# VLSI Design Internship Program

## (Batch: B2B DI 04 - SPI Design)

**Project:** APB interfaced SPI Master IP Core Design

SHAH MUKT BHAVIN
B.Tech ECE(2nd Year)
Reg.No.: 23BEC0012
VIT,Vellore

# Table of Contents


**Abstract** ..... 3

**Introduction** ..... 4

**Features** ..... 5

**SPI Block Diagram** ..... 6

**1. SPI Control Register 1 (SPICR1)** ..... 7

**2. SPI Control Register 2 (SPICR2)** ..... 7

**3. SPI Baud Rate Register (SPIBR)** ..... 8

**4. SPI Status Register (SPISR)** ..... 8

**5. SPI Data Register (SPIDR)** ..... 9

**APB interfaced SPI Master IP Block Diagrams** ..... 10

**Signal Descriptions** ..... 11

**Functionality** ..... 13

**Top Block** ..... 13

**Baud Rate Generator** ..... 18

**Shift Register** ..... 21

**APB Slave Interface** ..... 24

**SPI Slave Control Select** ..... 28

**Output Waveforms** ..... 31

**References** ..... 36

# Abstract

This report documents the design and Verilog HDL implementation of an APB-interfaced SPI (Serial Peripheral Interface) Master IP Core, developed in accordance with the Motorola S12SPIV3 specification. The SPI protocol is a widely adopted, full-duplex, synchronous serial communication standard, frequently used in embedded systems for interfacing microcontrollers with peripherals such as sensors, memory devices, and converters.

The core architecture comprises modular components including an APB-compliant slave interface, control logic, a programmable baud rate generator, and a fully synchronized shift register. It supports all four SPI modes (Mode 0 to Mode 3), offering flexibility through configurable clock polarity (CPOL), clock phase (CPHA), and data ordering (MSB/LSB first). The baud rate is precisely adjustable via prescaler and divider settings, enabling seamless adaptation to diverse peripheral requirements.

Special attention has been given to power management and error handling, with integrated support for low-power wait/stop modes and mode fault detection—mirroring the behavior of the S12SPIV3 reference implementation. Additionally, the design supports interrupt-driven communication, improving efficiency and reducing processor polling overhead.

This SPI Master IP Core delivers a scalable, synthesizable, and SoC-friendly solution for reliable serial communication, offering compatibility with AMBA-based systems and ensuring easy integration in both ASIC and FPGA workflows.

# Introduction

The Serial Peripheral Interface (SPI) is a widely utilized, full-duplex, synchronous communication protocol employed in embedded systems for efficient short-distance data exchange between a microcontroller and peripheral devices. Commonly used with memory modules, sensors, ADCs, DACs, and display drivers, SPI provides a high-speed, low-pin-count solution for serial data transfer.

In modern System-on-Chip (SoC) environments, integration and configurability are crucial. To address this, the presented project focuses on the design and Verilog implementation of a highly configurable SPI Master IP Core, interfaced via the Advanced Peripheral Bus (APB), in alignment with the AMBA APB v2.0 specification. The APB interface simplifies host-side interaction with the SPI core through standard memory-mapped registers, enabling seamless configuration and data transfer in AMBA-based systems.

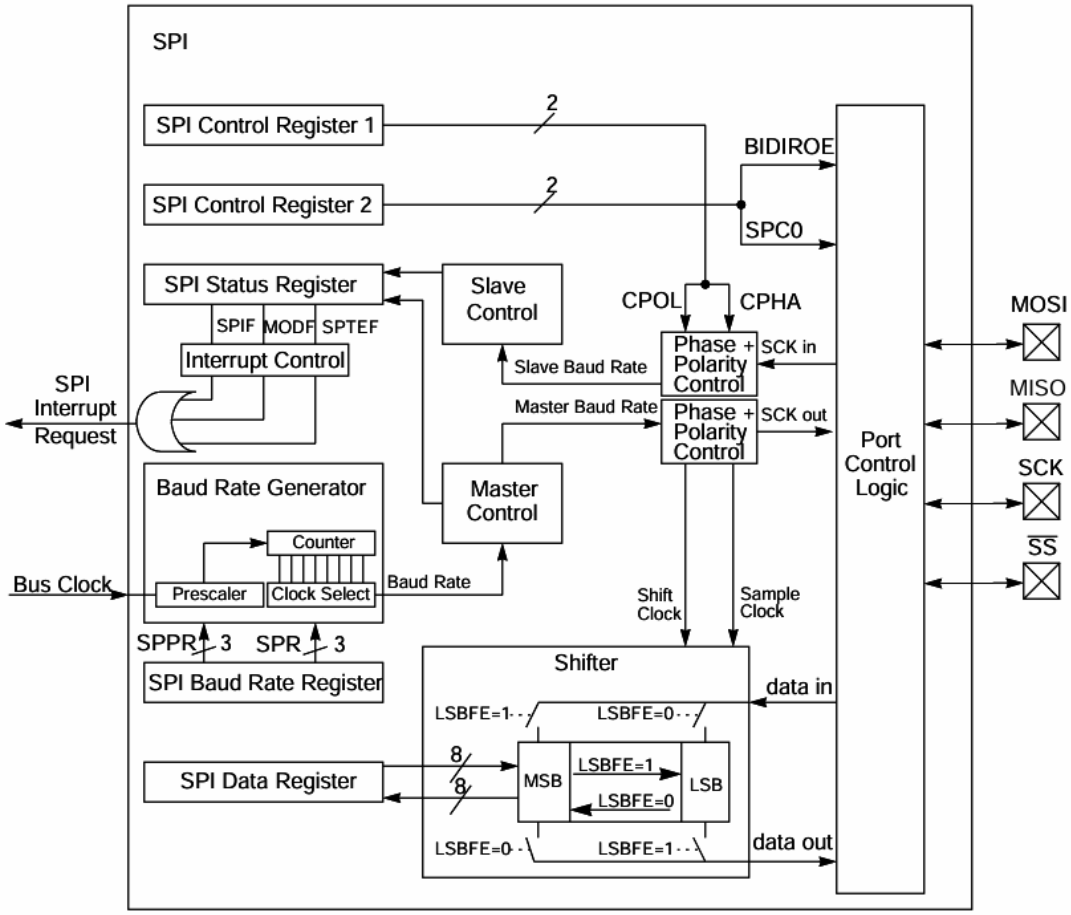
This IP core supports all four standard SPI modes (0–3), programmable baud rate control, and selectable data order (MSB-first or LSB-first), offering compatibility with a broad range of SPI-compliant devices. Inspired by the Motorola S12SPIV3 SPI Block Guide, the design incorporates robust features including low-power operation modes, interrupt capability, and bidirectional data handling.

Through a modular architecture—comprising an APB slave interface, shift register, control logic, and baud rate generator—this design ensures ease of integration, scalability, and reusability in both ASIC and FPGA implementations. This report elaborates on the architectural components, register-level interface, functional behavior, and simulation results of the proposed SPI Master IP Core.

# Features

- **Standard APB Slave Interface**
  - Compatible with AMBA APB v2.0 specification
  - Simple register-based configuration and control
  - Easy integration into SoC environments
- **Full-Duplex SPI Communication**
  - Simultaneous transmit and receive over MOSI and MISO lines
  - Supports 8-bit data frames (configurable extension possible)
- **Configurable SPI Modes (0 to 3)**
  - Support for all four SPI modes through programmable CPOL and CPHA
  - Enables communication with a wide range of SPI-compatible devices
- **Programmable Baud Rate**
  - User-defined clock divider allows dynamic control of SPI clock frequency
  - Enables adaptation to peripheral speed requirements
- **Master Mode Operation**
  - Generates SCLK and controls slave selection (SS line)
  - Manages the overall SPI transfer and timing
- **LSB First / MSB First Transfer Support**
  - Transfer order is configurable through a control register
  - Ensures compatibility with different peripheral formats

# SPI Block Diagram




## SPI Registers:

The SPI module includes the following key registers:

| **Register Name** | **Purpose** |
|---|---|
| **SPICR1** | Control Register 1 – Enables SPI, sets mode, and configures interrupts |
| **SPICR2** | Control Register 2 – Additional control settings: bidirectional mode, fault handling |
| **SPIBR** | Baud Rate Register – Sets the SPI clock frequency |
| **SPISR** | Status Register – Indicates transmission status, buffer state, and fault flags |
| **SPIDR** | Data Register – Used for data input/output (8-bit) |

### 1. SPI Control Register 1 (SPICR1)

| Bit | Name | Description |
|---|---|---|
| 7 | **SPIE** | Enables interrupt request when SPIF or MODF is set. |
| 6 | **SPE** | SPI System Enable: Activates SPI and connects port pins. |
| 5 | **SPTIE** | Transmit interrupt enable (when SPTEF is set). |
| 4 | **MSTR** | Selects Master (1) or Slave (0) mode. |
| 3 | **CPOL** | Clock polarity (Idle state of SCK): High if 1, Low if 0. |
| 2 | **CPHA** | Clock phase: Controls sampling edge. |
| 1 | **SSOE** | Slave Select Output Enable (Master mode only). |
| 0 | **LSBFE** | Transfers LSB first if set, MSB first if cleared. |

Note: Changing mode bits during a transfer **aborts** the operation and forces SPI to idle.

**Register Address: $___0**

| | Bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | Bit 0 |
|---|---|---|---|---|---|---|---|---|
| R<br>W | SPIE | SPE | SPTIE | MSTR | CPOL | CPHA | SSOE | LSBFE |
| Reset: | 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 |

---

### 2. SPI Control Register 2 (SPICR2)

| Bit | Name | Description |
|---|---|---|
| 7-5 | — | Reserved |
| 4 | **MODFEN** | Mode Fault Enable. Allows SPI to detect bus contention. |
| 3 | **BIDIROE** | Bidirectional Output Enable: Controls direction of single I/O pin. |
| 2 | **SPISWAI** | Stop SPI in Wait Mode: Saves power during CPU wait state. |
| 1 | **SPC0** | Serial Pin Control: Enables bidirectional data pin (MOMI/SISO). |
| 0 | — | Reserved |

**SPC0 = 1** → Enables **bidirectional** data on one pin. Controlled by **BIDIROE**.

**Register Address: $___1**

| | Bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | Bit 0 |
|---|---|---|---|---|---|---|---|---|
| R | 0 | 0 | 0 | MODFEN | BIDIROE | 0 | SPISWAI | SPC0 |
| W | | | | | | | | |
| Reset: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

= Reserved

---

### 3. SPI Baud Rate Register (SPIBR)

| Bits | Name | Description |
|---|---|---|
| 7-6 | — | Reserved |
| 5-3 | **SPPR[2:0]** | Baud Rate Preselector |
| 2-0 | **SPR[2:0]** | Baud Rate Divider selector |

**Baud Rate Formula:**

Baud Rate Divisor = (SPPR+1) x $(2^{(SPR+1)})$

Baud Rate=$\frac{Bus\ Clock}{Baud\ Rate\ Divisor}$

Example: For SPPR=001 and SPR=010, divisor = 2 × 4 = 8 → Baud rate = 25 MHz / 8 = 3.125 MHz

**Register Address: $___2**

| | Bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | Bit 0 |
|---|---|---|---|---|---|---|---|---|
| R | 0 | SPPR2 | SPPR1 | SPPR0 | 0 | SPR2 | SPR1 | SPR0 |
| W | | | | | | | | |
| Reset: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

= Reserved

---

### 4. SPI Status Register (SPISR)

| Bit | Name | Description |
|---|---|---|
| 7 | **SPIF** | SPI Transfer Complete Flag: Set after 8-bit transfer finishes. Cleared by reading SPISR then SPIDR. |
| 6 | — | Reserved |
| 5 | **SPTEF** | Transmit Empty Flag: Set if data register is ready. Cleared by reading SPISR (with SPTEF=1) then writing to SPIDR. |
| 4 | **MODF** | Mode Fault: Set if SS goes low in Master mode (with MODFEN=1). |
| 3-0 | — | Reserved |

**Register Address: $___3**

| | Bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | Bit 0 |
|---|---|---|---|---|---|---|---|---|
| R | SPIF | 0 | SPTEF | MODF | 0 | 0 | 0 | 0 |
| W | | | | | | | | |
| Reset: | 0 | 0 | 1 | 0 | 0 | 0 | 0 | 0 |

= Reserved

---

### 5. SPI Data Register (SPIDR)

| Bit | Name | Description |
|---|---|---|
| 7-0 | Data | 8-bit data register for transmission (write) or reception (read). |

- **Write**: Queues data for transmission.
- **Read**: Retrieves received data after SPIF = 1.

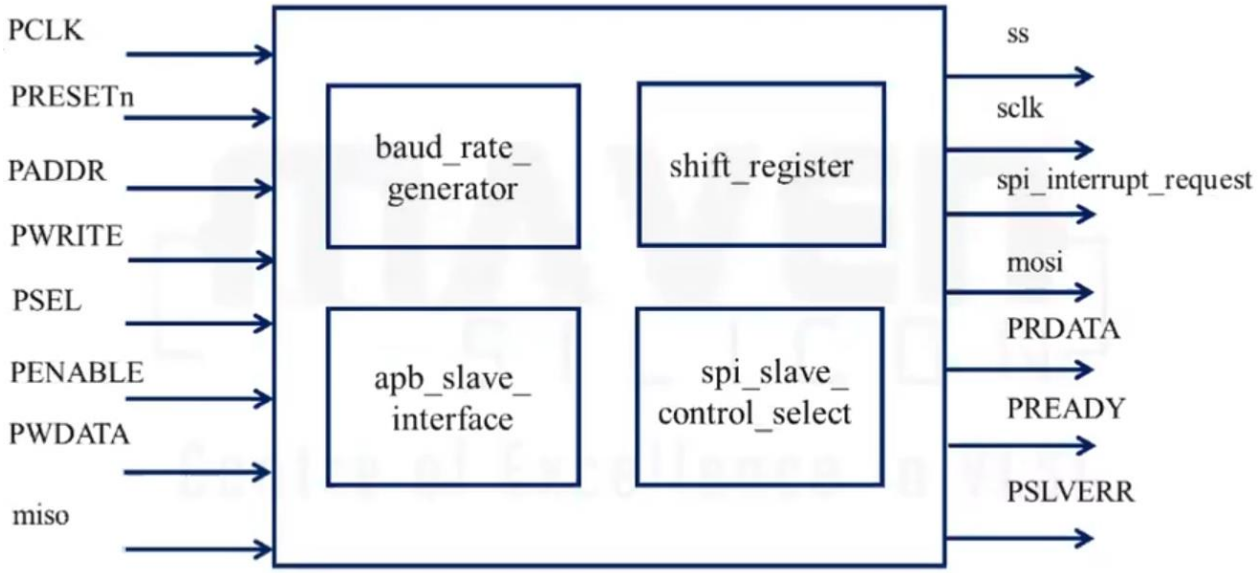
Double-buffered: Data is first stored in a shift register, then transferred to SPIDR.

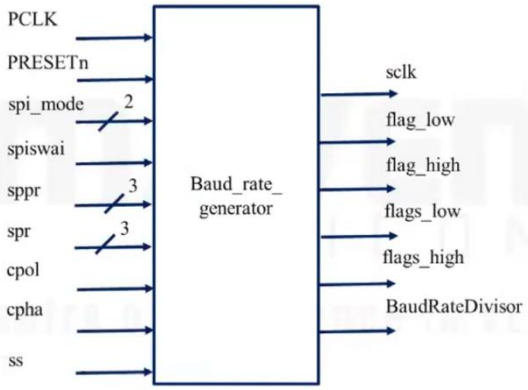
**Register Address: $___5**

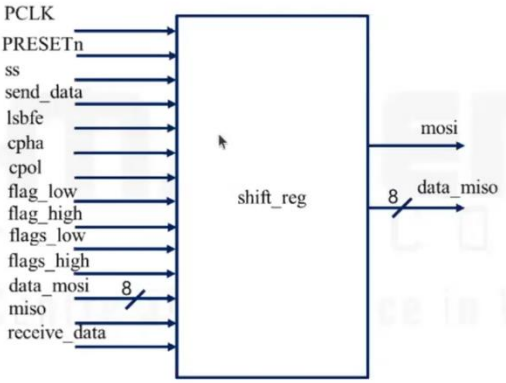
| | Bit 7 | 6 | 5 | 4 | 3 | 2 | 1 | Bit 0 |
|---|---|---|---|---|---|---|---|---|
| R<br>W | Bit 7 | 6 | 5 | 4 | 3 | 2 | 2 | Bit 0 |
| Reset: | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

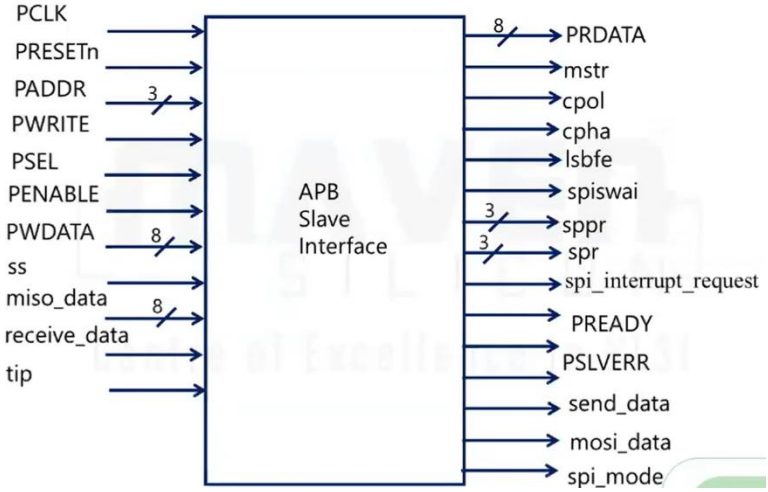
---

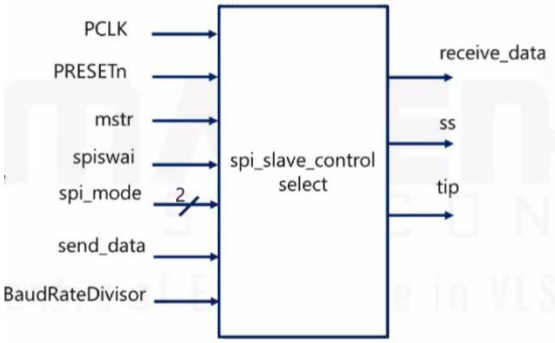
# APB interfaced SPI Master IP Block Diagrams



# Signal Descriptions

| Signal | Width | Direction | Signal Description |
|---|---|---|---|
| PCLK | 1 | Input | System Clock |
| PRESETn | 1 | Input | Active Low Asynchronus Reset Signal |
| PADDR | 3 | Input | APB Address bus |
| PSEL | 1 | Input | APB Slave Select Signal |
| PENABLE | 1 | Input | PENABLE indicates the second and subsequent APB transfer |
| PWRITE | 1 | Input | Indicates the Write (1) or Read(0). |
| PWDATA | 8 | Input | The PWDATA is write data bus. |
| PRDATA | 8 | Output | The PRDATA is read data bus. |
| PREADY | 1 | Output | PREADY is used to extend an APB Transfer by Core |
| PSLVERR | 1 | Output | PSLVERR is used to indicate Transfer Error. |
| ss | 1 | Input | Slave Select (Active Low Signal) |
| spi_interrupt_request | 1 | Output | SPI Interrupt Request |

| Signal | Width | Direction | Signal Description |
|---|---|---|---|
| receive_data | 1 | Input | Indicates the transfer of data(MISO data) from Shifter register to data register. |
| miso_data | 8 | Input | MISO Data from shift register |
| tip | 1 | Input | Transfer In Progress |
| send_data | 1 | Output | Indicates the transfer of data from data register to shifter register |
| mstr | 1 | Output | Indicates Master(1) or Slave (0). |
| cpol | 1 | Output | Clock Polarity |
| cpha | 1 | Output | Clock Phase |
| lsbfe | 1 | Output | LSB First Enable |
| spiswai | 1 | Output | SPI Stop in Wait Mode |
| mosi_data | 8 | Output | MOSI Data from Data Register to Shift Register |
| spi_mode | 2 | Output | SPI Mode(Run Mode, Wait Mode and Stop Mode) |
| spr | 3 | Output | SPI Baud Rate Selection Bits |
| sppr | 3 | Output | SPI Baud Rate Preselection Bit |

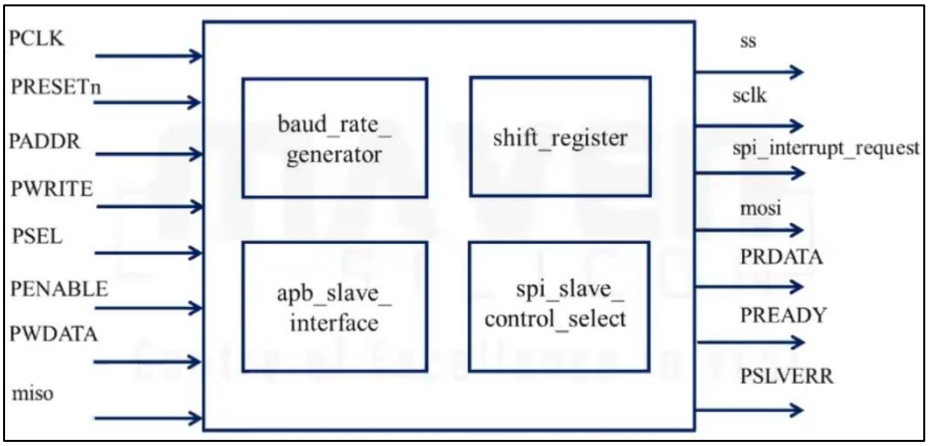
| Signal | Width | Direction | Signal Description |
|---|---|---|---|
| PCLK | 1 | Input | System Clock |
| PRESETn | 1 | Input | Active Low Asynchronus Reset Signal |
| cpol | 1 | Input | Clock Polarity |
| spiswai | 1 | Input | SPI Stop in Wait Mode |
| spi_mode | 2 | Input | SPI Mode(Run Mode, Wait Mode and Stop Mode) |
| spr | 3 | Input | SPI Baud Rate Selection Bits |
| sppr | 3 | Input | SPI Baud Rate Preselection Bit |
| ss | 1 | Input | Slave Select Active Low Signal |
| sclk | 1 | Output | Serial Clock |
| BaudRateDivisor | 12 | Output | Baud Rate Divisor |
| cphase | 1 | Input | Clock Phase |
| flag_low | 1 | Output | Flag to indicate when to receive the miso data when both cphase and cpol are high or low. |
| flag_high | 1 | Output | Flag to indicate when to receive the miso data when either of the cphase or cpol is high. |
| flags_low | 1 | Output | Flag to indicate when to send the mosi data when both cphase and cpol are high or low. |
| flags_high | 1 | Output | Flag to indicate when to send the mosi data when either of the cphase or cpol is high. |

| Signal | Width | Direction | Signal Description |
|---|---|---|---|
| PCLK | 1 | Input | APB Clock |
| PRESETn | 1 | Input | Active Low Asynchronus Reset Signal |
| ss | 1 | Input | Slave Select (Active Low Clock) |
| receive_data | 1 | Input | Indicates the transfer of data(MISO data) from Shifter register to data register. |
| data_miso | 8 | Output | MISO Data from shift register |
| send_data | 1 | Input | Indicates the transfer of data from data register to shifter register |
| miso | 1 | Input | Master In Slave Out. |
| cpol | 1 | Input | Clock Polarity |
| cpha | 1 | Input | Clock Phase |
| lsbfe | 1 | Input | LSB First Enable |
| mosi | 1 | Output | Master Out Slave In |
| data_mosi | 8 | Input | MOSI Data from Data Register to Shift Register |
| flag_low | 1 | Input | Flag to indicate when to receive the miso data when both cphase and cpol are high or low. |
| flag_high | 1 | Input | Flag to indicate when to receive the miso data when either of the cphase or cpol is high. |
| flags_low | 1 | Input | Flag to indicate when to send the mosi data when both cphase and cpol are high or low. |
| flags_high | 1 | Input | Flag to indicate when to send the mosi data when either of the cphase or cpol is high. |

| Signal | Width | Direction | Signal Description |
|---|---|---|---|
| PCLK | 1 | Input | System Clock |
| PRESETn | 1 | Input | Active Low Asynchronus Reset Signal |
| mstr | 1 | Input | SPI is in Master Mode |
| send_data | 1 | Input | Send Data from Data Register |
| spiswai | 1 | Input | SPI Stop in Wait Mode |
| spi_mode | 2 | Input | SPI Mode(Run Mode, Wait Mode and Stop Mode) |
| ss | 1 | Output | Slave Select Active Low Signal |
| baudratedivisor | 12 | Input | Baud Rate Divisor |
| tip | 1 | Output | Transfer in Pregress |
| receive_data | 1 | Output | Receive Data to Data Register |

# Functionality

## Top Block




The **APB-Interfaced SPI Master IP Core** is a highly modular, synthesizable, and configurable hardware design that facilitates seamless serial communication between a host processor and one or more peripheral devices using the widely adopted **Serial Peripheral Interface (SPI)** protocol. Designed to integrate efficiently with **AMBA-based systems**, this core uses the **Advanced Peripheral Bus (APB)** as its primary control interface. Through the APB, the host processor can easily configure operational parameters and control the SPI core using standard memory-mapped register accesses, ensuring compatibility with a broad range of SoC platforms.

The architecture of this IP core is organized into four major functional blocks: the **apb_slave_interface**, **spi_slave_control_select**, **baud_rate_generator**, and **shift_register**. Each of these blocks performs a specific function essential to the correct operation of the SPI master:

- The **apb_slave_interface** is responsible for managing all APB transactions. It interprets address (PADDR), data (PWDATA), and control signals (PWRITE, PSEL, PENABLE) from the host processor and generates appropriate response signals (PRDATA, PREADY, PSLVERR). This block ensures correct data routing between the processor and the SPI registers, enabling

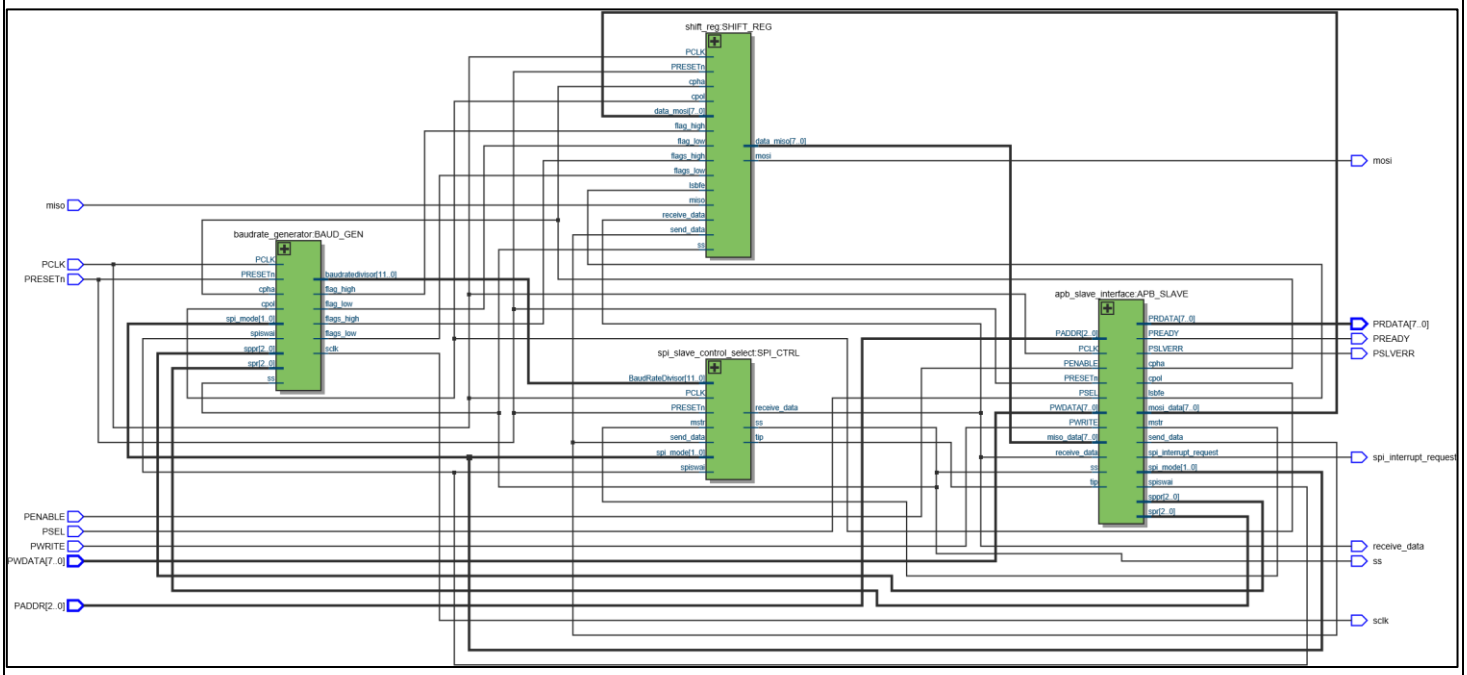
configuration of parameters such as clock polarity (CPOL), clock phase (CPHA), data order (MSB/LSB first), slave select behavior, and interrupt enable.

- The **spi_slave_control_select** module governs the control logic required for SPI operation. It manages the **slave select signal (ss)**, oversees **data transfer sequencing**, and handles **receive data logic**. It also interacts with internal flags to ensure that SPI communication adheres to protocol requirements and generates the **spi_interrupt_request** signal to notify the processor upon completion of a transfer or when specific conditions are met.
- The **baud_rate_generator** dynamically generates the SPI serial clock (sclk) by dividing the input system clock (PCLK) according to a programmable baud rate divisor. This feature allows the SPI master to operate at various speeds, making it suitable for interfacing with peripherals that have different timing requirements. The flexibility of the baud rate adjustment also enhances the core's adaptability across applications ranging from low-speed sensors to high-speed memory devices.
- The **shift_register** block is at the heart of the SPI data path. It handles the parallel-to-serial conversion of outgoing data (transmitted via the mosi line) and the serial-to-parallel conversion of incoming data received on the miso line. The operation of this block is fully synchronized with the sclk signal and respects the selected SPI mode (defined by CPOL and CPHA), ensuring accurate data transfer in all supported configurations.

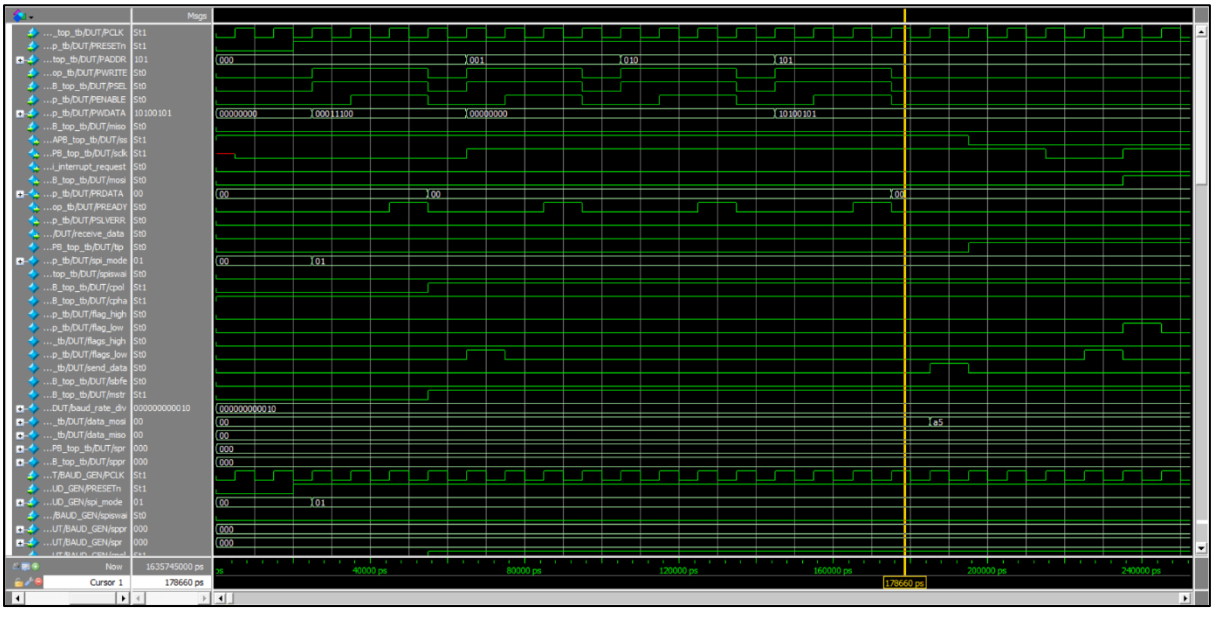
On the output side, the core provides essential SPI signals including **mosi (Master Out Slave In)**, **sclk (Serial Clock)**, **ss (Slave Select)**, and optionally, **spi_interrupt_request** to signal events such as transmission completion. Internally, all data read and write operations are orchestrated using standard APB handshaking mechanisms, involving the **PWDATA**, **PRDATA**, **PREADY**, and **PSLVERR** signals.

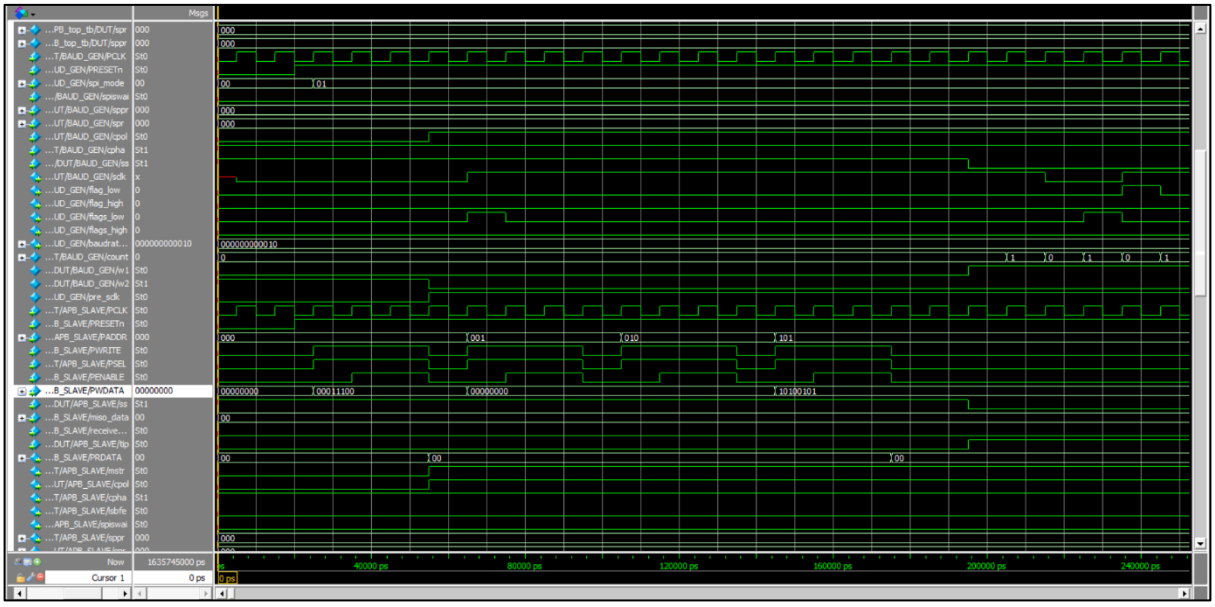
In summary, this APB-Interfaced SPI Master IP Core offers a robust, scalable, and reusable solution for embedding SPI functionality into SoC designs. Its clean separation of control and data logic, compatibility with APB systems, and support for flexible clock configurations make it ideal for a wide range of embedded applications where reliable and efficient serial communication is critical.

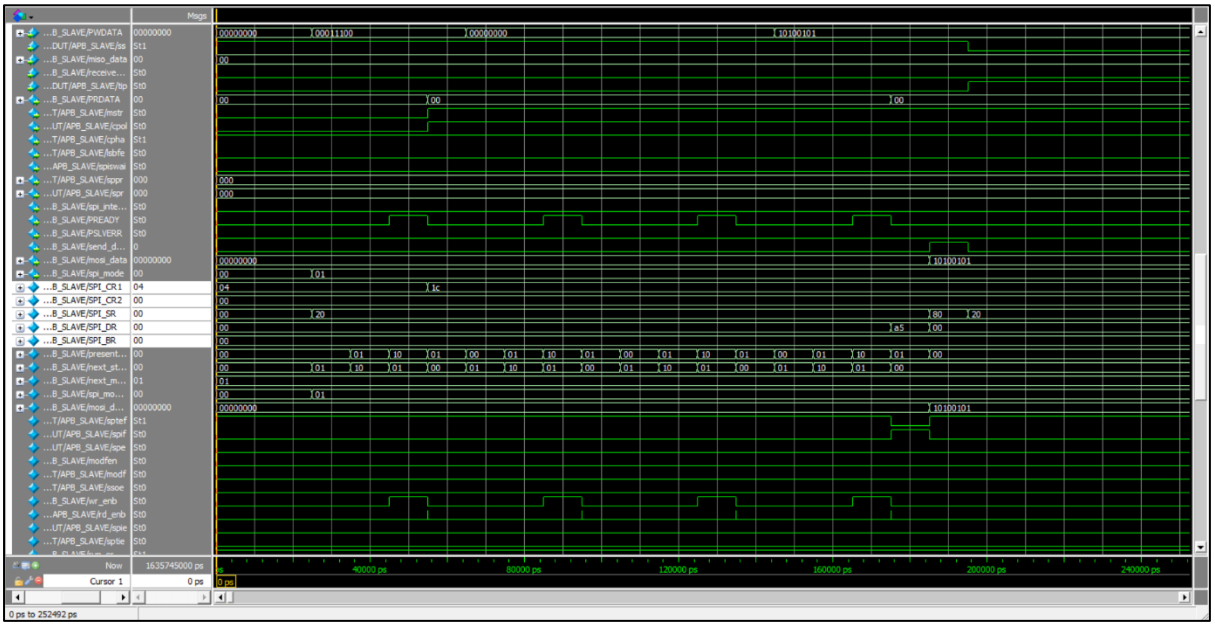
## ***Synthesis:***

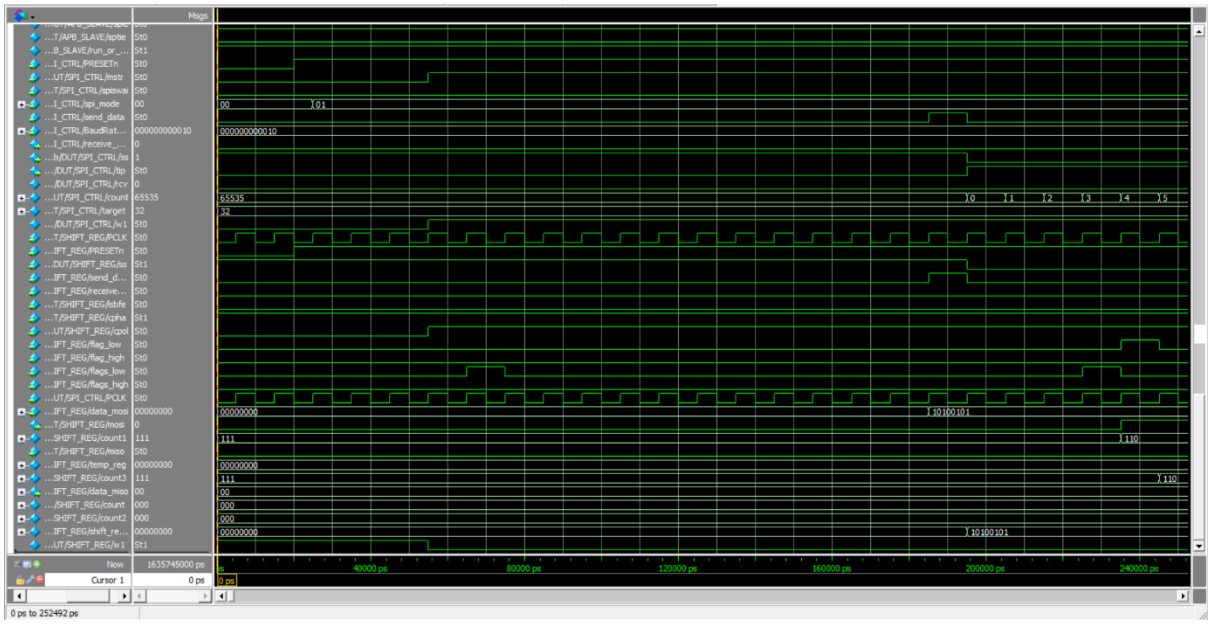


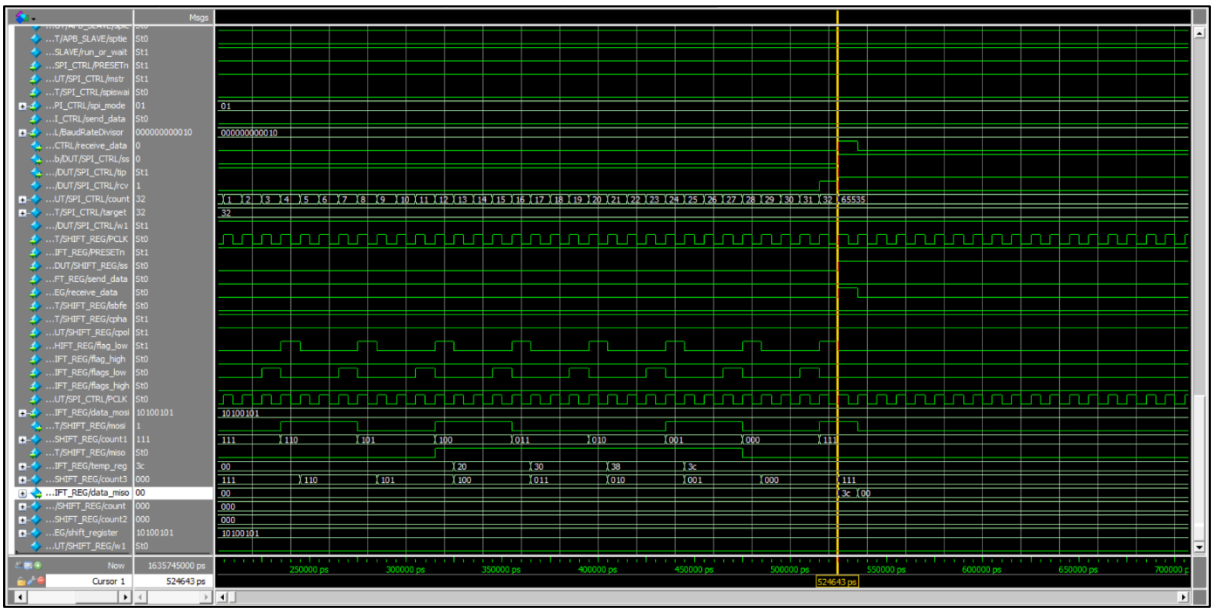

## ***Waveform:***

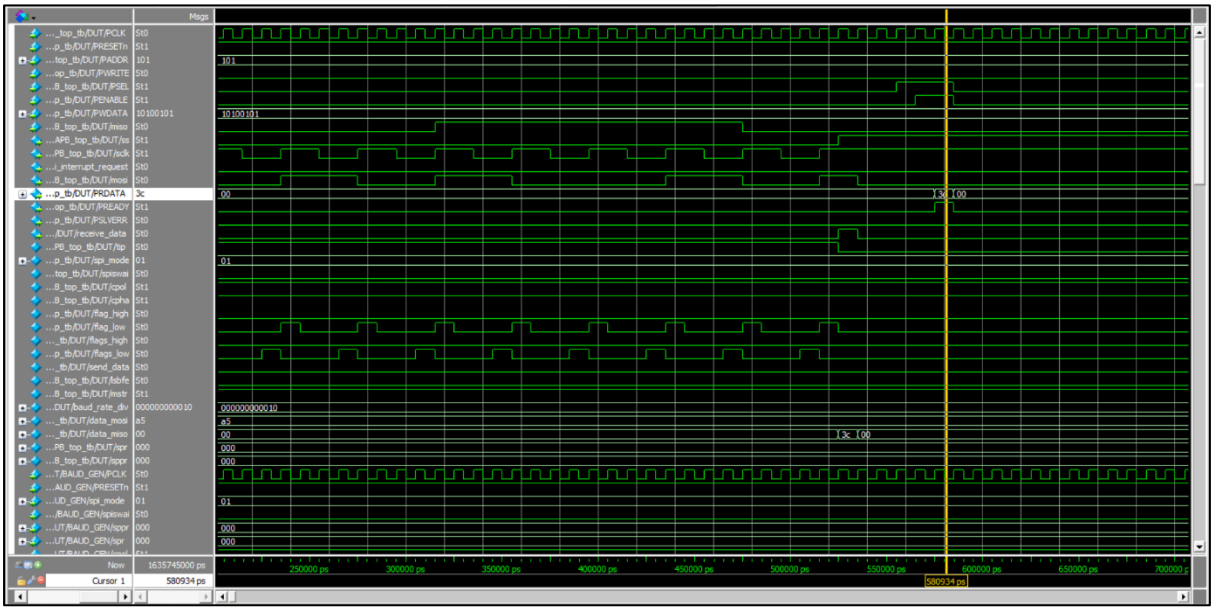




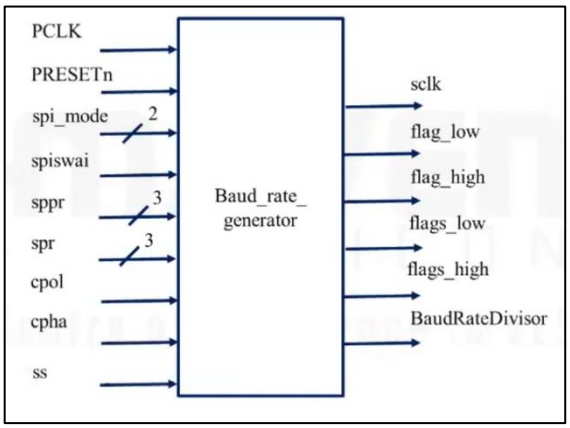








# Baud Rate Generator




The **Baud Rate Generator** is a fundamental and timing-critical component of the SPI Master IP Core. It is responsible for generating the **serial clock (sclk)**, which acts as the backbone for synchronization between the master and slave devices during SPI communication. This module ensures that data transmission and reception are precisely timed according to the selected SPI mode and desired communication speed.

At its core, the Baud Rate Generator takes in the high-frequency **system clock (PCLK)** and produces a slower clock signal through a programmable frequency division mechanism. The division ratio is calculated based on two register-configurable fields: **spr (Serial Clock Rate)** and **sppr (Serial Clock Prescaler)**. These values allow for a wide range of baud rate settings, enabling the core to adapt to various peripheral devices with different timing requirements. The output clock (sclk) is thus derived from the formula involving spr and sppr, offering precise and flexible baud rate control.

In addition to frequency division, the Baud Rate Generator incorporates critical **SPI mode configuration signals**, namely **cpol (Clock Polarity)** and **cpha (Clock Phase)**. These parameters determine the behavior of the sclk signal relative to data sampling and shifting, aligning with the four standard SPI modes (Mode 0 to Mode 3). By handling these parameters internally, the generator ensures that the clock signal aligns correctly with the timing expectations of the connected slave devices.

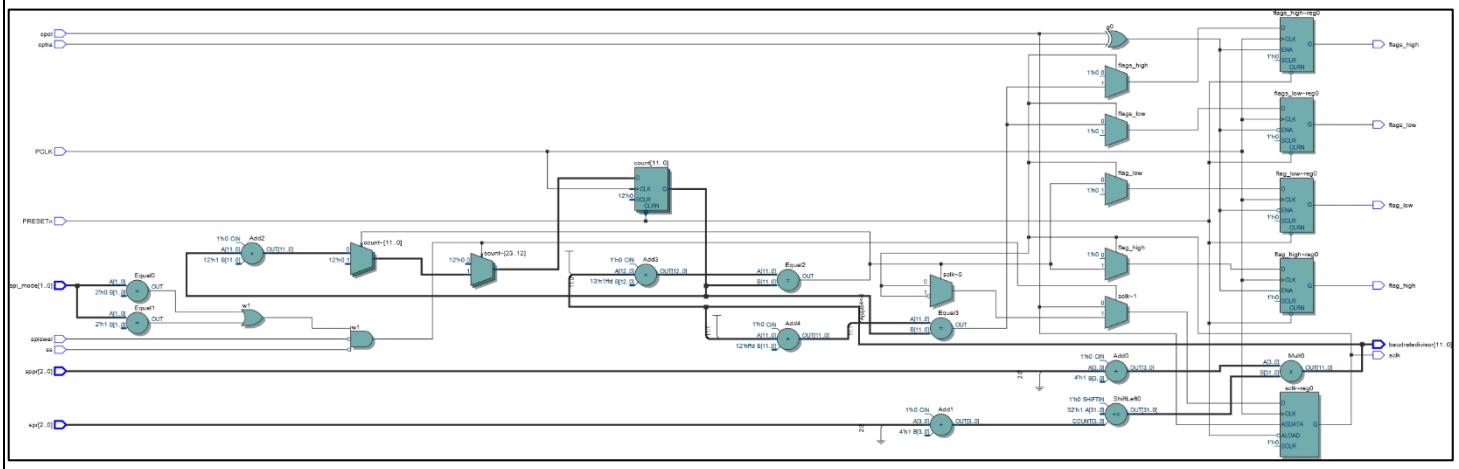
The module also interfaces with **spi_mode** and **spiswai** control signals, which support higher-level control functions such as **SPI enable/disable states** and **software-initiated wait or idle modes**. These signals allow dynamic clock gating or disabling of sclk to conserve power or prevent unintended communication during inactive periods. Additionally, the **ss (Slave Select)** signal is used to gate the output clock—ensuring that the sclk is only active during an ongoing SPI transaction, thereby avoiding interference with other devices on the SPI bus.

To support the internal timing and coordination of other modules—particularly the **shift register**—the Baud Rate Generator produces a set of timing control outputs: **flag_low**, **flag_high**, **flags_low**, and **flags_high**. These **edge-sensitive flags** indicate critical timing points within the clock cycle, such as when a rising or falling edge occurs. These signals are essential for precisely timing the data shift and sample operations, ensuring correct serialization on the mosi line and accurate sampling from the miso line.

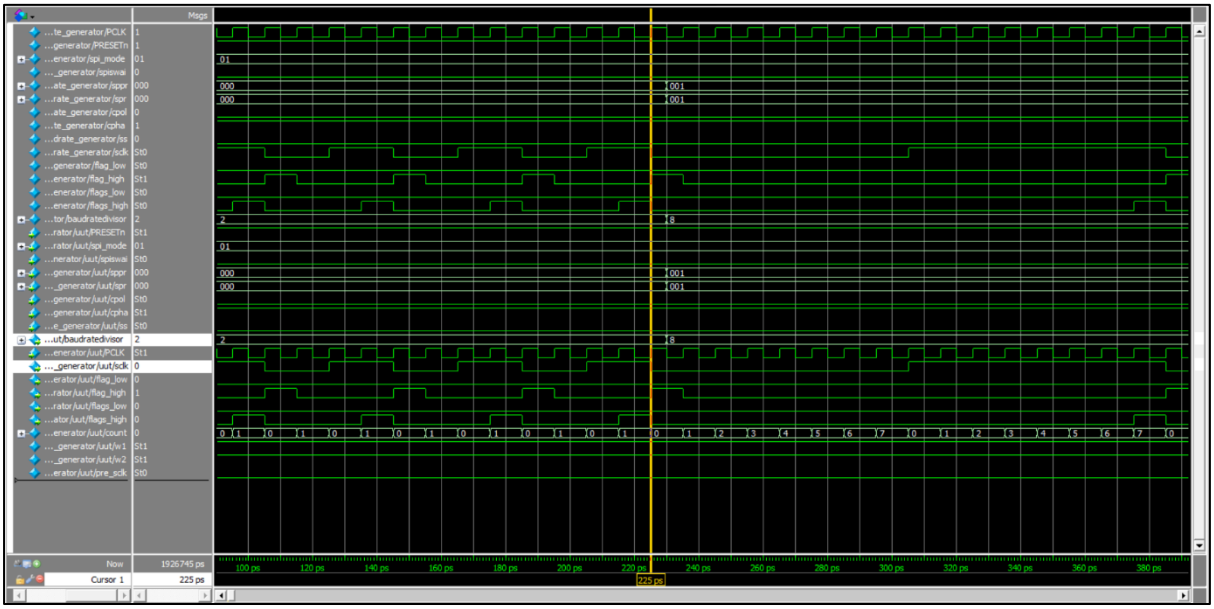
Furthermore, the module outputs the **BaudRateDivisor**, which represents the currently applied division factor based on spr and sppr. This value can be referenced by other logic blocks—such as the SPI control or interrupt modules—to synchronize their operation with the generated clock frequency.

In conclusion, the Baud Rate Generator is not merely a frequency divider—it is an intelligent, mode-aware timing engine that orchestrates the clocking behavior of the entire SPI Master Core. It ensures robust, reliable, and protocol-compliant communication by generating precisely timed clock signals tailored to both user configurations and SPI standard specifications.

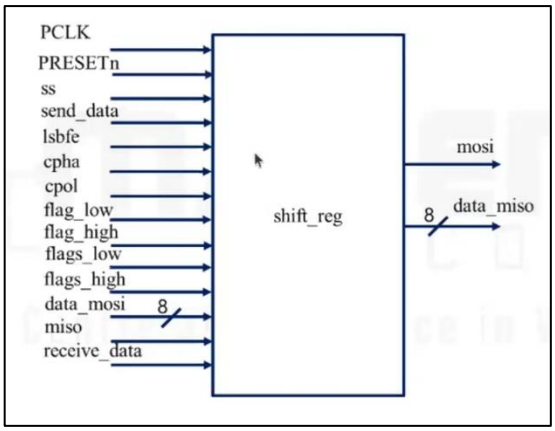
## ***Synthesis***:



## ***Waveform:***



# Shift Register




The **Shift Register** module is a core functional unit within the SPI Master IP Core, tasked with handling the **bit-level serialization and deserialization of data** during SPI transactions. It plays a pivotal role in enabling full-duplex communication between the master and connected slave devices by supporting **simultaneous transmission and reception** of data bits over the MOSI (Master Out Slave In) and MISO (Master In Slave Out) lines.

This module operates in **synchrony with the system clock (PCLK)** and adheres to the system's **active-low reset (PRESETn)** signal to ensure proper initialization and fault recovery. It integrates tightly with the control and timing infrastructure of the SPI core, especially the outputs from the **Baud Rate Generator**, which include the clock edge indicators: **flag_high**, **flag_low**, **flags_high**, and **flags_low**. These flags correspond to specific rising or falling edges of the SPI clock (sclk) and are essential in determining precisely when bits are shifted out or sampled in, according to the SPI protocol's timing specifications.

The shift register accepts parallel data input via the **data_mosi** signal—an 8-bit wide bus containing the byte to be transmitted. Based on the SPI clock timing, this byte is **serially shifted out bit-by-bit** through the **mosi** output line. The order in which bits are transmitted is governed by the **lsbfe** control signal (Least Significant Bit First Enable), which determines whether data is sent **LSB-first** or **MSB-**

**first**, allowing compatibility with diverse SPI peripheral requirements.

To initiate a transmission, the **send_data** signal is asserted. This triggers the module to begin shifting out the data_mosi bits on the mosi line in accordance with the SPI mode settings defined by **cpol** (clock polarity) and **cpha** (clock phase). These mode bits dictate the precise relationship between the SPI clock and data transitions, ensuring compatibility with all four SPI modes (0 through 3). The data transmission is fully synchronized with the edge-sensitive flags from the Baud Rate Generator, allowing precise control over when data bits are updated or sampled.

Simultaneously, the module captures incoming bits from the **miso** line whenever the **receive_data** signal is active. Each incoming bit is stored in a temporary internal register as it arrives. Once all 8 bits have been received, the completed byte is assembled and presented on the **data_miso** output bus, making it available to the system for further processing or storage.

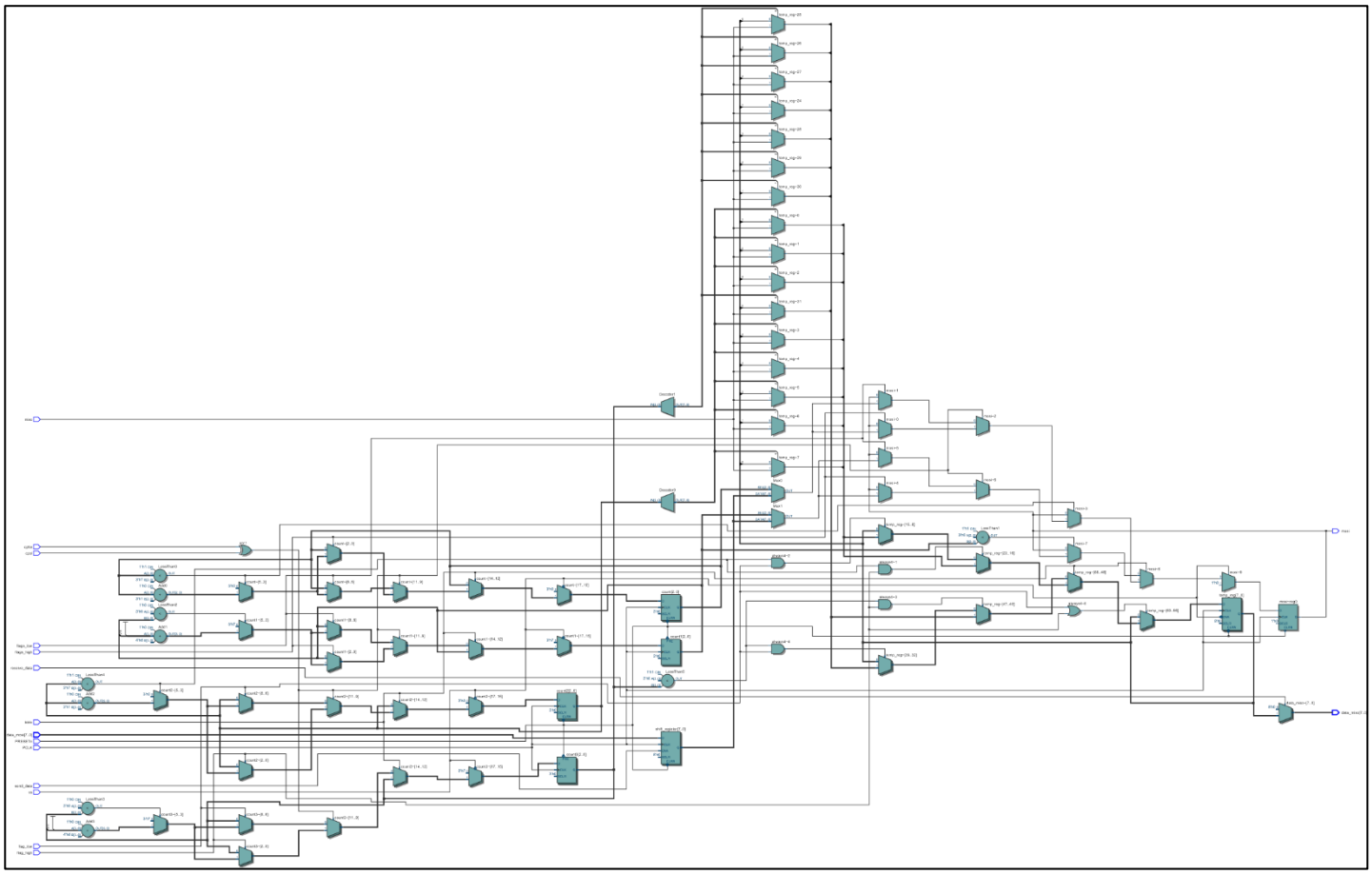
The **ss (slave select)** input is another critical control signal that ensures the shift register only becomes active when a valid SPI transaction is in progress. When ss is asserted (typically active-low), it indicates that a slave device is selected and that transmission and reception should proceed. This gating mechanism prevents unintended data exchange and ensures that the SPI bus remains free from conflicts or erroneous activity when no slave is selected.

Because of its ability to **simultaneously transmit and receive data**, the shift register inherently supports **full-duplex operation**, a key advantage of the SPI protocol. This means that while the master sends data to the slave, it can concurrently receive data in return during the same clock cycles—resulting in efficient and high-throughput communication.

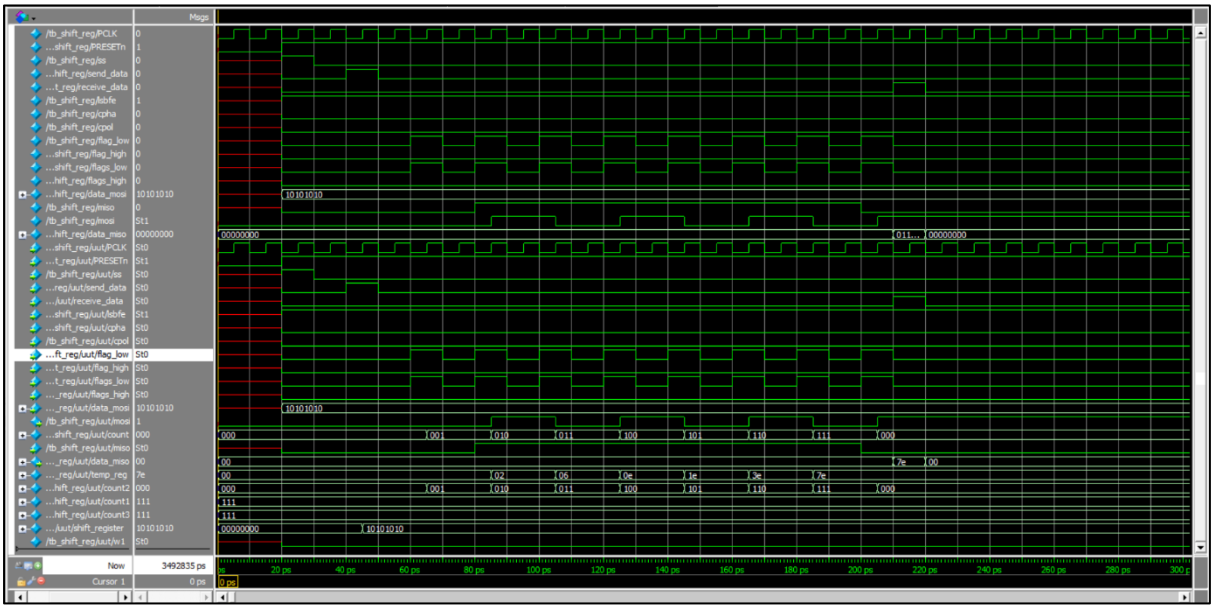
In summary, the **Shift Register** module is a fundamental building block of the SPI Master IP Core, enabling precise, mode-compliant, and synchronized serial data exchange. Its integration with control,

timing, and data management signals ensures reliable and configurable operation across a variety of SPI devices and use cases.

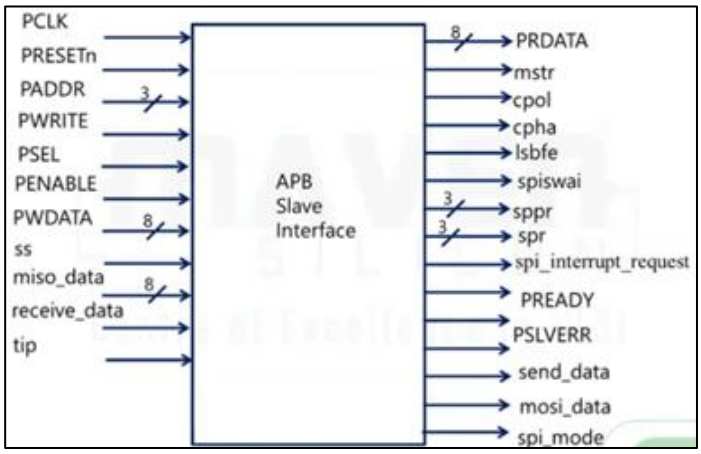
## ***Synthesis:***



## ***Waveform:***



# APB Slave Interface




The **APB Slave Interface** module is a crucial subsystem within the SPI Master IP Core architecture, serving as the **bridge between the AMBA Advanced Peripheral Bus (APB)** and the internal logic of the SPI controller. This module enables seamless communication and control from a host processor or bus master, allowing configuration, status monitoring, and data exchange through **standardized memory-mapped APB transactions**.

This interface adheres strictly to the APB protocol, receiving all necessary control and data signals, including **PCLK** (APB clock), **PRESETn** (active-low reset), **PADDR** (address bus), **PWRITE** (read/write indicator), **PSEL** (slave select), **PENABLE** (transfer enable), and **PWDATA** (write data bus). During a transaction, these signals are used to decode which internal SPI register is being accessed, whether the operation is a read or write, and what data is to be transferred.

Upon decoding a valid write access, the APB interface updates internal configuration registers or control signals. These include:

- **mstr** – enables or disables the SPI master functionality.
- **cpol** and **cpha** – define the SPI clock polarity and phase, respectively, configuring the SPI mode (0 to 3).
- **lsbfe** – selects the bit ordering (LSB-first or MSB-first) during data transmission.

- **spi_mode** – controls overall SPI operational state, including enabling or disabling data transfers.
- **spiswai** – allows SPI communication to pause or enter a wait/sleep state, contributing to power-efficient operation.
- **spr** and **sppr** – specify the baud rate division factors used by the Baud Rate Generator to produce the desired SPI clock frequency.
- **mosi_data** – an 8-bit register that holds the data to be transmitted on the MOSI line.
- **send_data** – a control signal that triggers the start of a new SPI transaction.

In **read operations**, the module supplies data back to the APB master via the **PRDATA** output bus. This data can include the received SPI byte (captured via the miso_data line), current configuration settings, or status flags—such as whether a transfer is in progress or completed.

The interface also continuously monitors several **internal SPI signals** to manage status reporting and system-level coordination. These include:

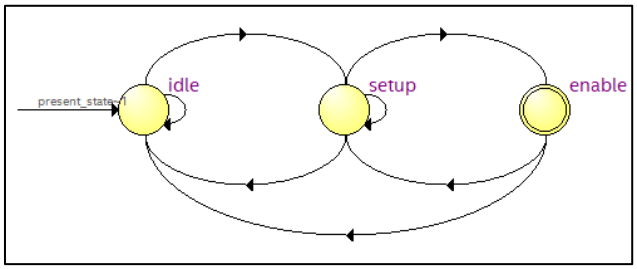
- **ss (Slave Select)** – indicates whether an SPI transaction is currently active.
- **tip (Transfer In Progress)** – reflects whether data shifting is ongoing.
- **receive_data** – signifies that valid data has been received and is ready to be read.
- **miso_data** – the data received via the MISO line, which is captured by the shift register and made available for APB reads.

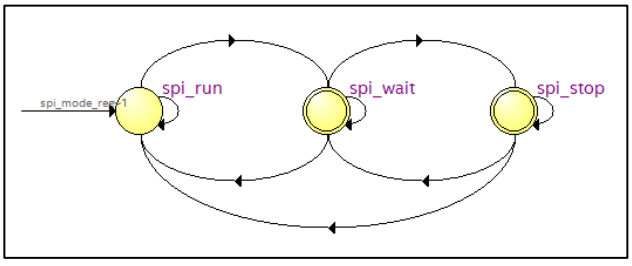
To maintain APB protocol compliance, the module generates appropriate response signals:

- **PREADY** – signals to the APB master that a transaction (read or write) has completed and the interface is ready for the next access.
- **PSLVERR** – indicates any error conditions, such as invalid addresses or access timing issues.
- **spi_interrupt_request** – an optional interrupt output that can be used to notify the processor when a transfer has completed or when new data is available, supporting efficient **interrupt-driven communication** rather than relying solely on polling.

By abstracting and encapsulating all SPI configuration and status reporting within the APB framework, this module provides a **robust, scalable, and flexible interface** for integrating the SPI Master Core into larger SoC platforms. It ensures **tight coupling with AMBA-compliant host processors**, facilitating clean and efficient register-based communication while maintaining full compatibility with industry-standard APB designs.

In summary, the **APB Slave Interface** module enables reliable and structured control of the SPI core through familiar memory-mapped transactions, ensuring that both configuration and data operations are intuitive and tightly synchronized with the broader system architecture.

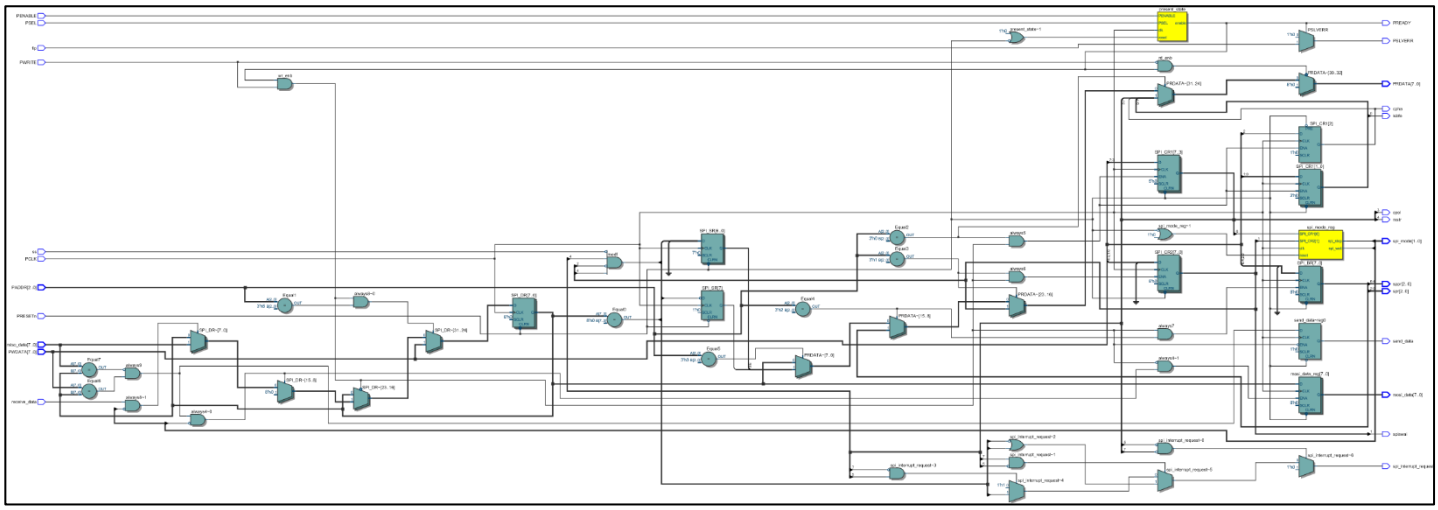


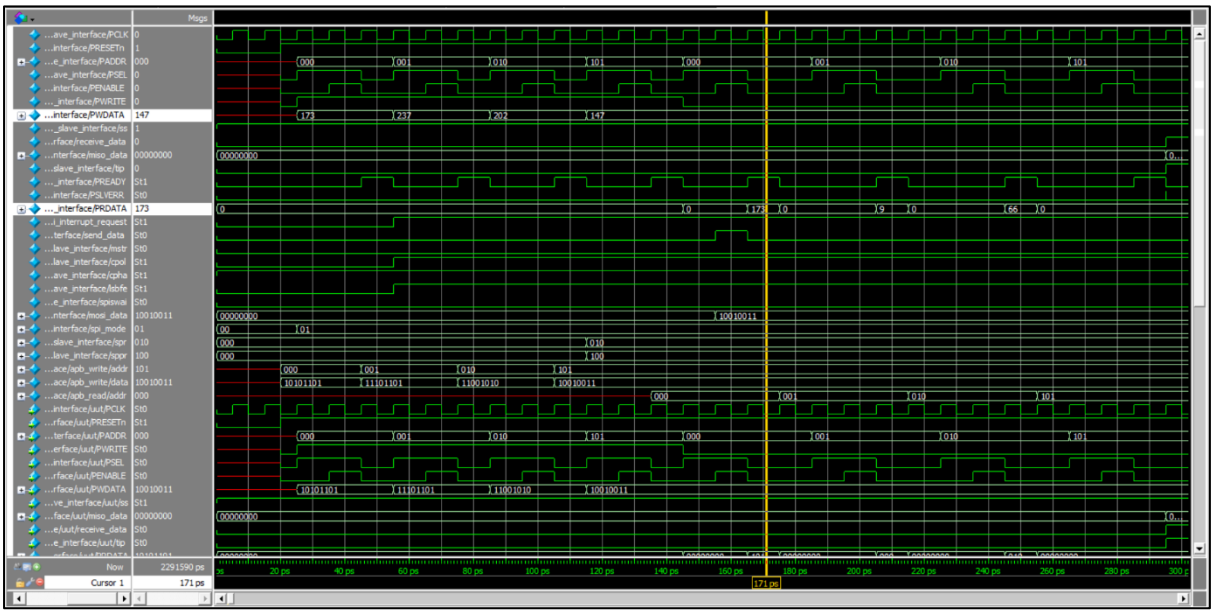

| | Source State | Destination State | Condition |
|---|---|---|---|
| 1 | enable | setup | (PSEL) |
| 2 | enable | idle | (!PSEL) |
| 3 | idle | setup | (PSEL).(!PENABLE) |
| 4 | idle | idle | (!PSEL) + (PSEL).(PENABLE) |
| 5 | setup | enable | (PSEL).(PENABLE) |
| 6 | setup | setup | (PSEL).(!PENABLE) |
| 7 | setup | idle | (!PSEL) |

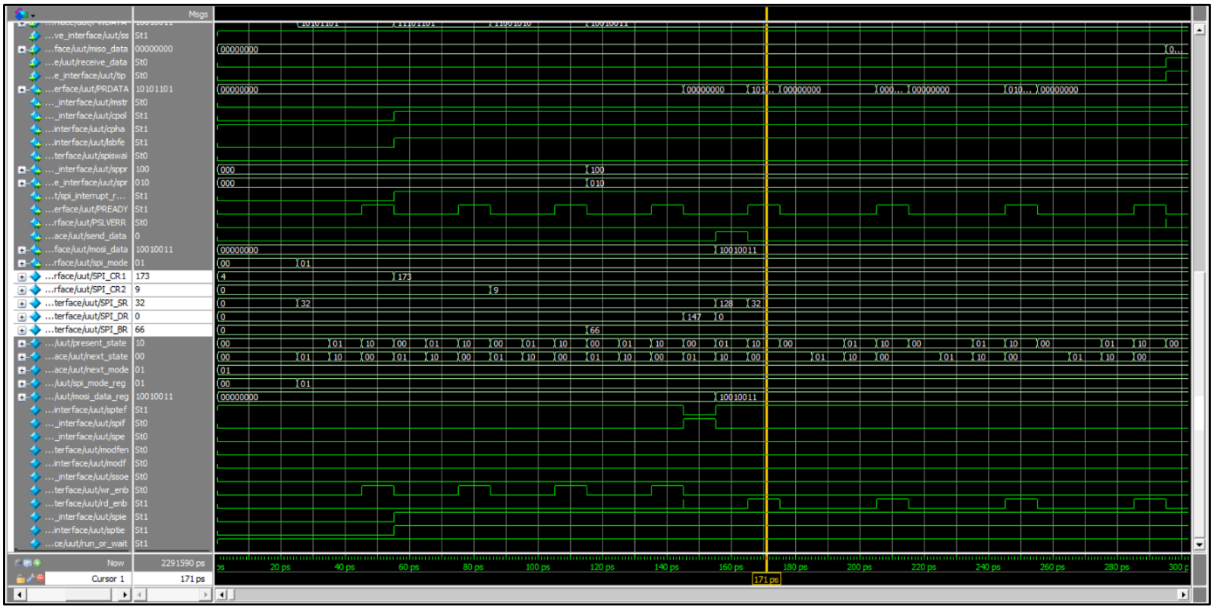
| | Source State | Destination State | Condition |
|---|---|---|---|
| 1 | spi_run | spi_wait | (!SPI_CR1[6]) |
| 2 | spi_run | spi_run | (SPI_CR1[6]) |
| 3 | spi_stop | spi_wait | (!SPI_CR2[1]) |
| 4 | spi_stop | spi_run | (SPI_CR1[6]).(SPI_CR2[1]) |
| 5 | spi_stop | spi_stop | (!SPI_CR1[6]).(SPI_CR2[1]) |
| 6 | spi_wait | spi_wait | (!SPI_CR1[6]).(!SPI_CR2[1]) |
| 7 | spi_wait | spi_run | (SPI_CR1[6]) |
| 8 | spi_wait | spi_stop | (!SPI_CR1[6]).(SPI_CR2[1]) |

## ***Synthesis:***

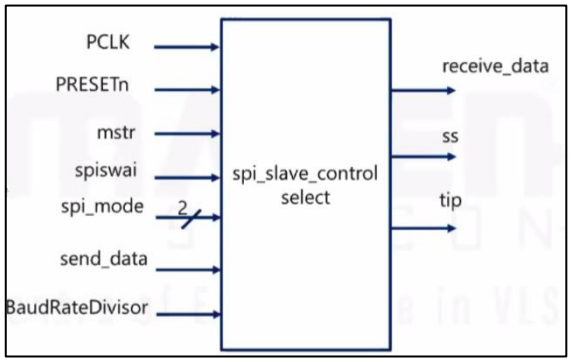


## ***Waveform:***





# SPI Slave Control Select




The **SPI Slave Control Select** module is a **central control unit** within the SPI Master IP Core that is responsible for orchestrating the overall flow of SPI communication. It handles the **initiation, control, and termination** of each SPI transaction and ensures all operations are carried out in compliance with the SPI protocol. Acting as a **state-driven controller**, it generates the required internal and external control signals to manage the timing and coordination of data exchanges with SPI slave devices.

This module operates synchronously with the **system clock (PCLK)** and is initialized using an **active-low reset (PRESETn)**, which resets all internal states and ensures a known startup condition. Once initialized, the module becomes responsive to a set of **configuration and control inputs** that govern its behavior and determine the specifics of the SPI communication.

**Key Control Inputs**

- **mstr (Master Enable):** This signal indicates whether the SPI module is operating in master mode. It must be asserted to allow the control logic to drive the ss (slave select) signal and control the SPI clock generation.
- **spiswai (SPI Sleep/Wait Control):** Used to manage low-power or idle states, this input can temporarily halt SPI operations during sleep modes or software-controlled wait states, aiding in system-wide power management strategies.

- **spi_mode (2-bit):** Defines the SPI operation mode (Mode 0 to Mode 3), which is determined by the combination of **clock polarity (CPOL)** and **clock phase (CPHA)**. These bits influence the alignment of data sampling and shifting in synchronization with the SPI clock (sclk).

**Transfer Initiation and Timing**

The SPI Slave Control Select module listens for the **send_data** signal, which acts as a trigger for starting a new SPI transfer. Once this signal is asserted, the module takes charge of the full transaction lifecycle. It uses the **BaudRateDivisor** input—computed by the Baud Rate Generator based on programmable prescaler values—to regulate timing throughout the transfer. This divisor ensures that the data is shifted and sampled at the correct intervals, maintaining protocol compliance and timing accuracy.

**Control of Slave Select and Status Monitoring**

One of the most important responsibilities of this module is to manage the **ss (Slave Select)** signal. The ss output is asserted (typically driven low) at the beginning of a transfer to activate the target SPI slave device and remains asserted throughout the entire transaction. Once the transfer is completed, the ss signal is deasserted, ensuring the slave is cleanly deselected. This behavior ensures **proper chip select timing**, which is critical for correct operation of many SPI devices.

During the data transfer, the module asserts the **tip (Transfer In Progress)** signal. This real-time status output remains high for the duration of the transaction, allowing both the **shift register** and the **APB interface** to monitor the communication status. External software or hardware can also use tip to prevent overlapping transactions or manage timing-sensitive operations.

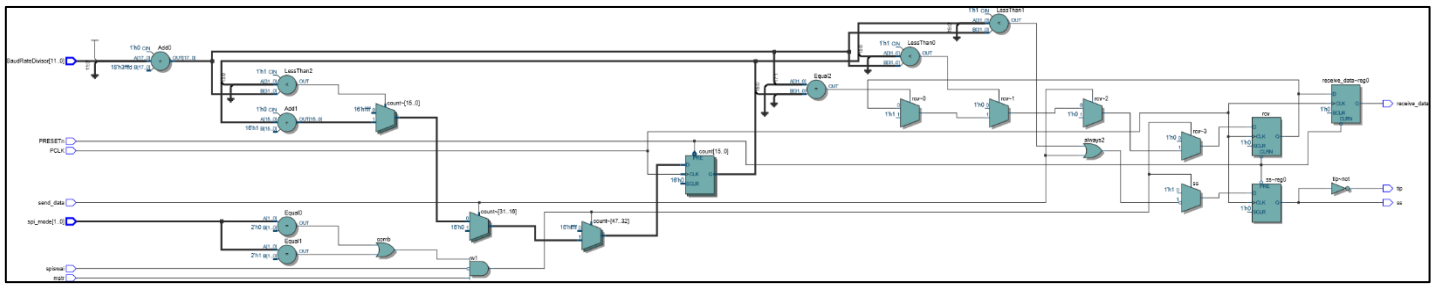
**Data Reception and Transfer Completion**

At the conclusion of the SPI transaction, especially in cases where the master expects a response or data from the slave, the module asserts the **receive_data** signal. This output informs the system that valid

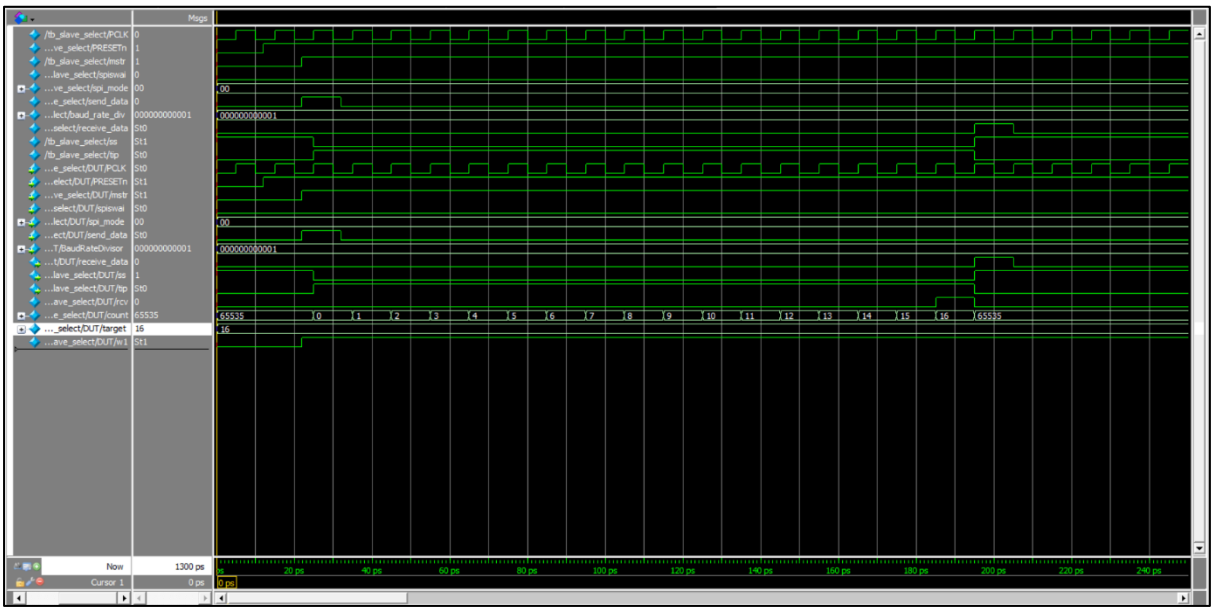
incoming data has been received via the miso line and is ready to be read from internal registers. This event can also trigger an interrupt (via spi_interrupt_request from another module) to notify the processor that data is available for processing.

By coordinating with other blocks—such as the **shift register**, **baud rate generator**, and **APB interface**—the SPI Slave Control Select module ensures **synchronized and reliable data flow**. It acts as the glue logic that binds together the timing, data, and control aspects of the SPI Master Core.

***Synthesis:***

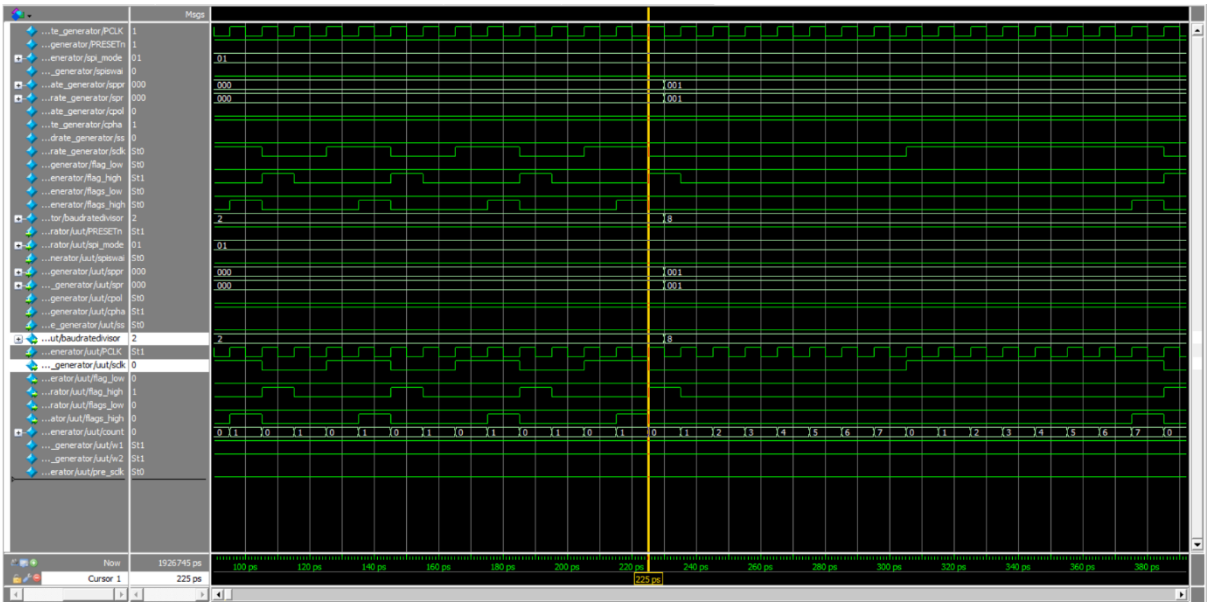


***Waveform:***

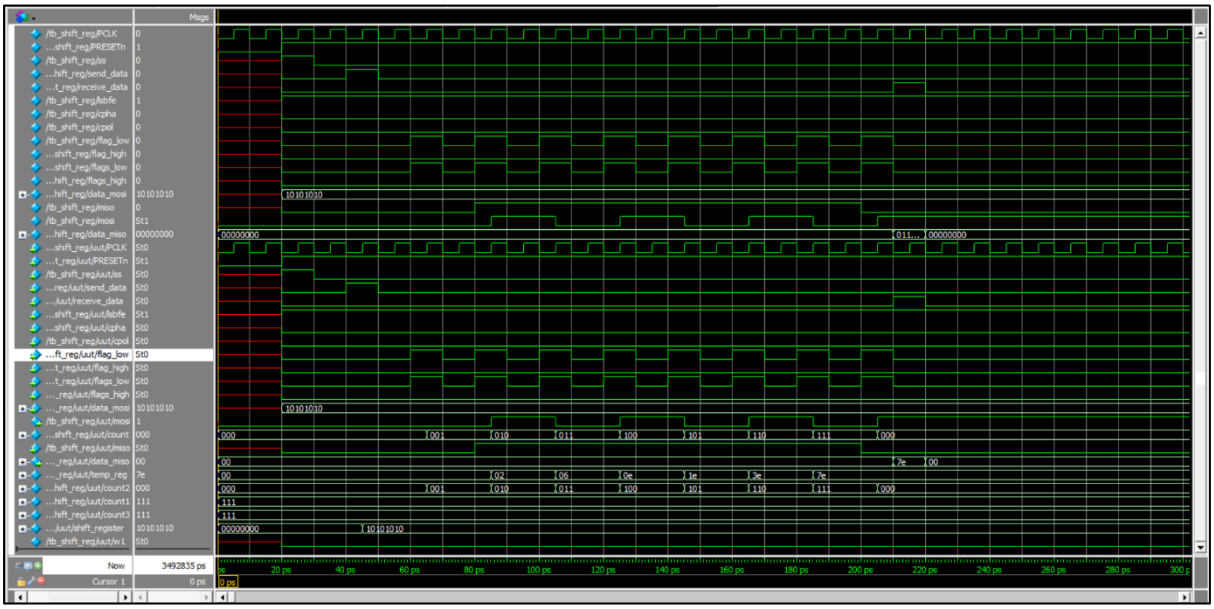


# Output Waveforms

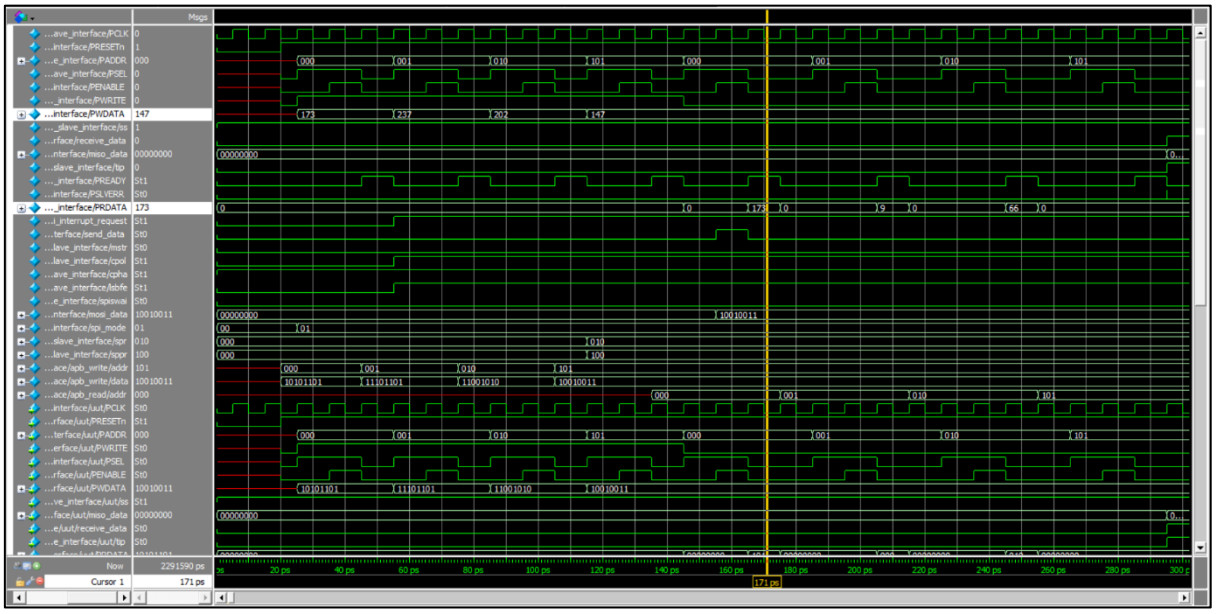
## Baud Rate Generator:

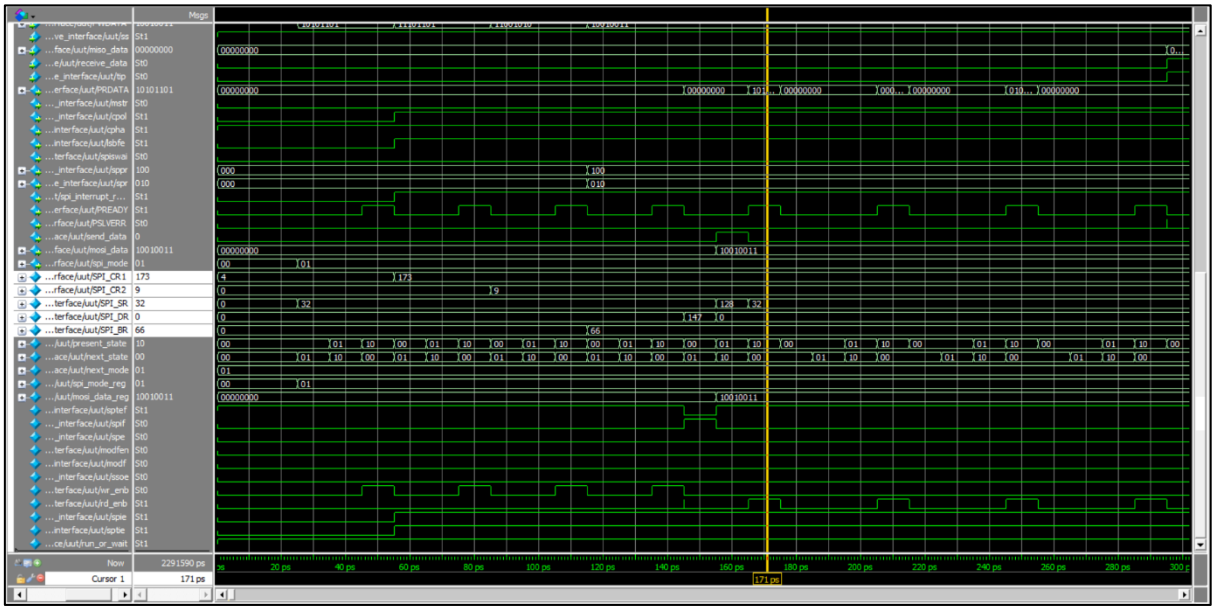


## Shift Register:

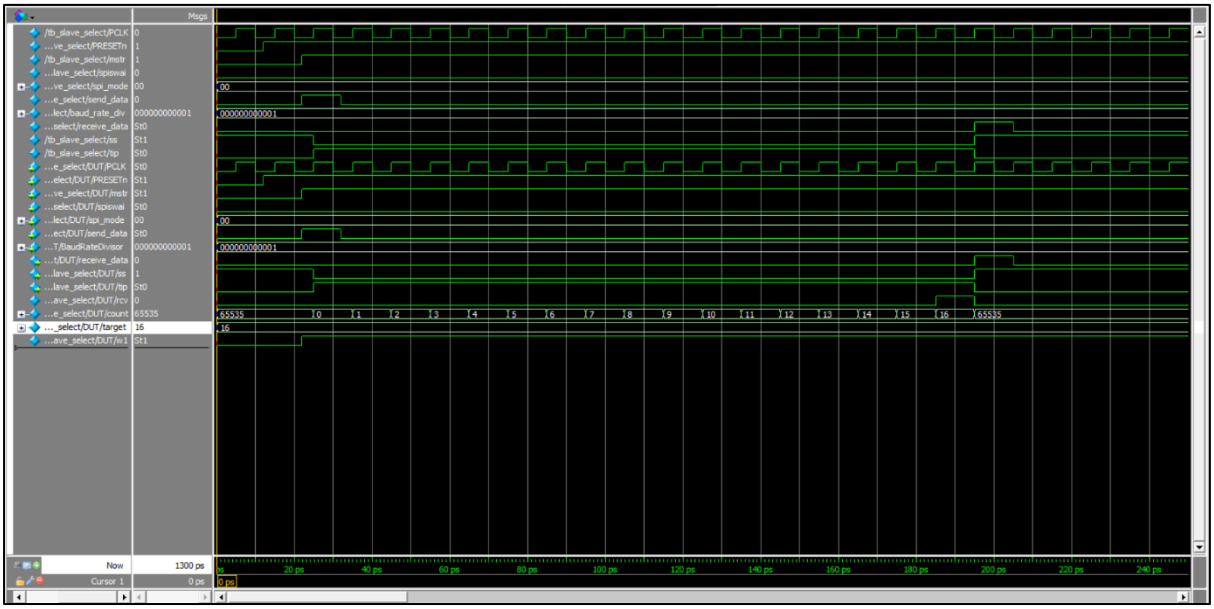


# APB Slave Interface:

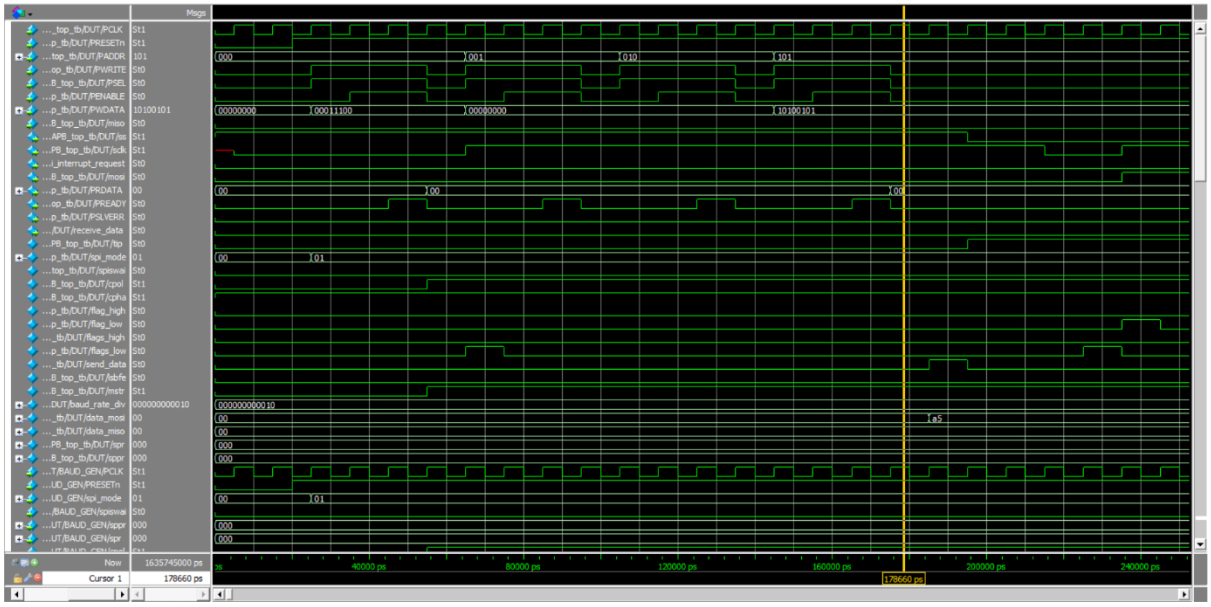


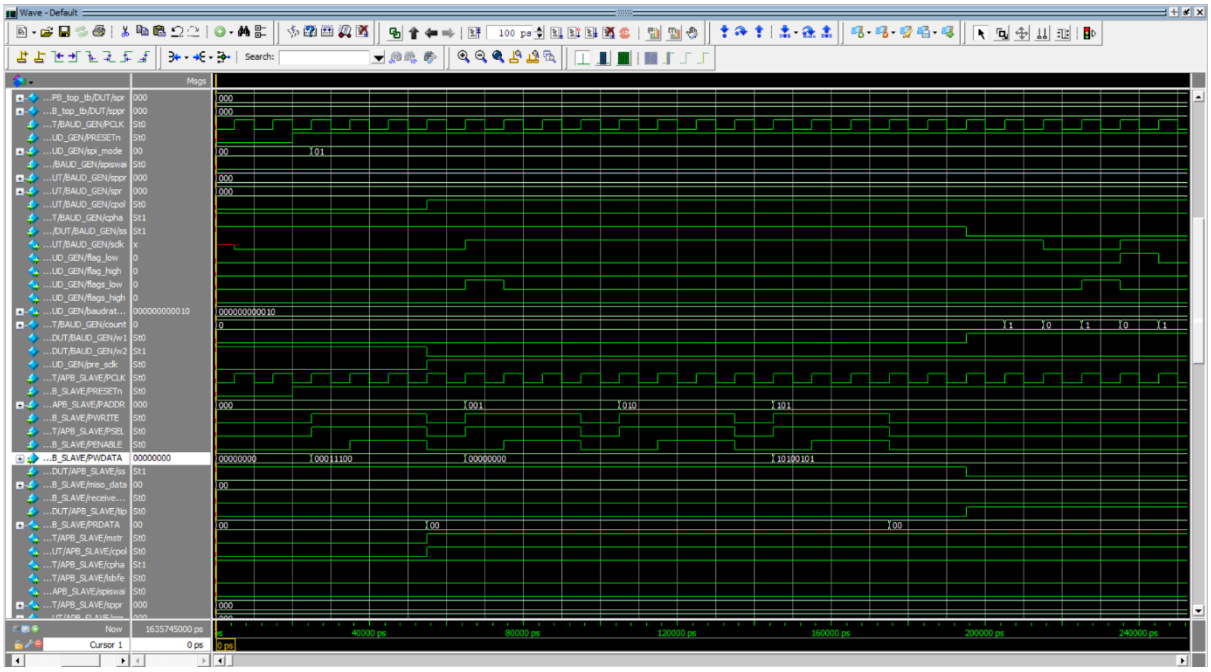


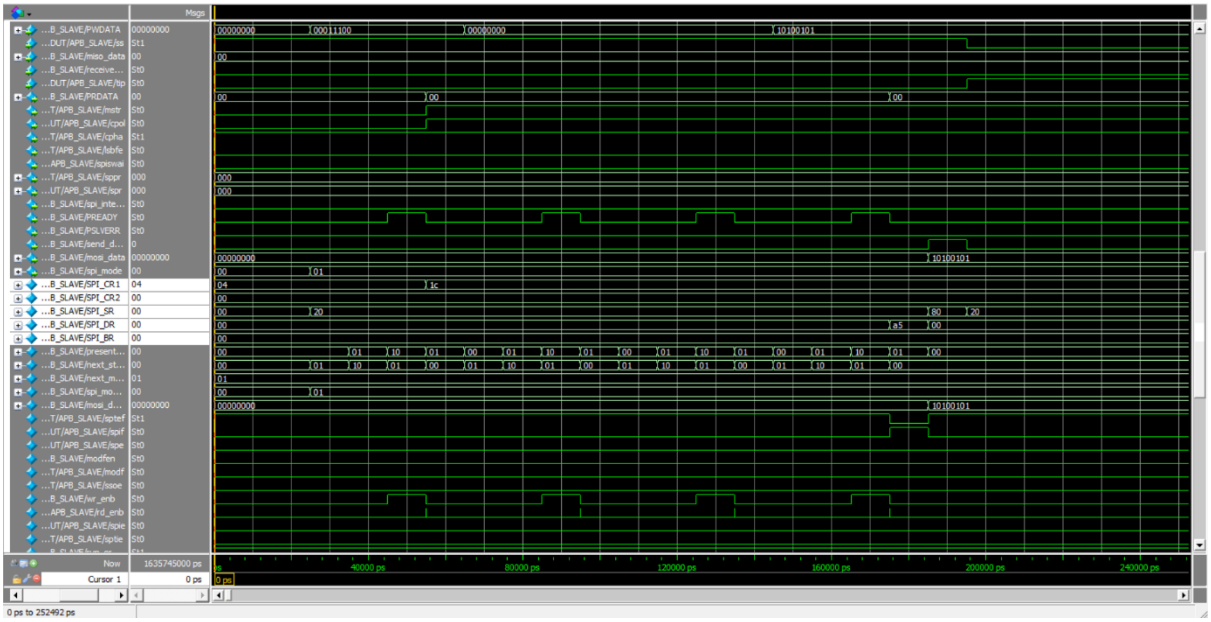
# SPI Slave Control Select:

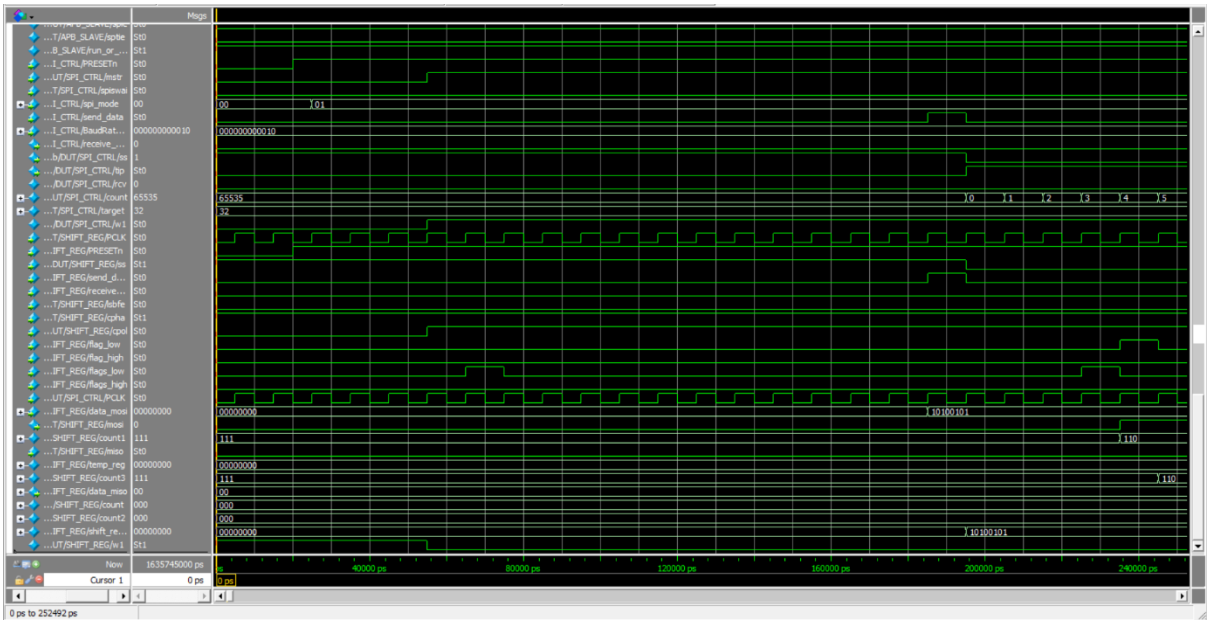


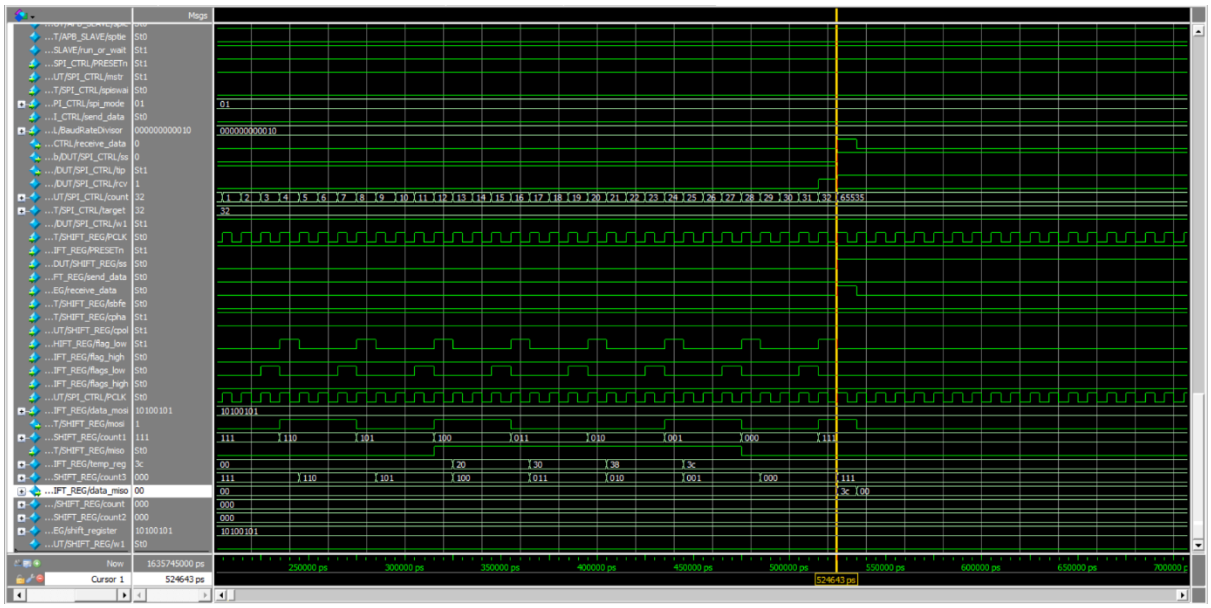
# Top Block:

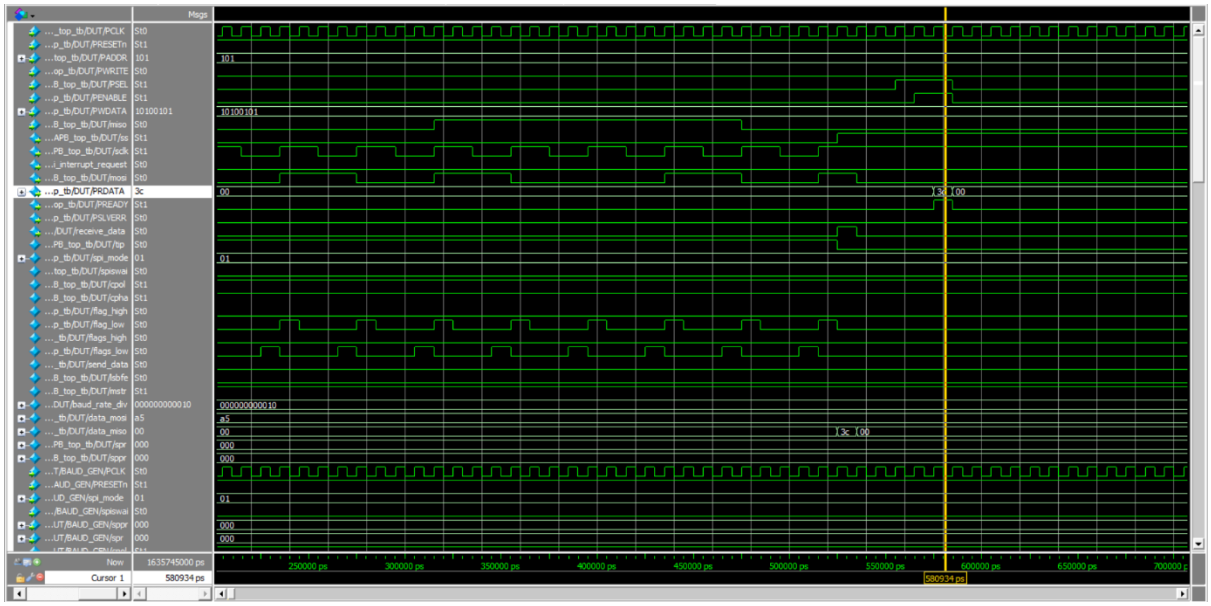



Wave - Default
Msgs
Now
1635745000 ps
Cursor 1
0 ps
40000 ps
80000 ps
120000 ps
160000 ps
200000 ps
240000 ps


Msgs
Now
1635745000 ps
Cursor 1
0 ps
40000 ps
80000 ps
120000 ps
160000 ps
200000 ps
240000 ps
0 ps to 252492 ps


Msgs
Now
1635745000 ps
Cursor 1
0 ps
40000 ps
80000 ps
120000 ps
160000 ps
200000 ps
240000 ps
0 ps to 252492 ps





# References

[1] Motorola Inc., *SPI Block Guide V03.06*, Document No. S12SPIV3/D, Rev. 04 Feb 2003.

[2] ARM Ltd., *AMBA 3 APB Protocol Specification*, Issue B, ARM IHI 0024B, 2nd Release, Aug. 2004.

[3] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*, 2nd ed., Pearson Education, 2003.